

OPERATING SYSTEM

Sessional Exam Study Notes

MCA Semester I | Nirma University | Units 1 to 6 of your sessional syllabus deck

These notes are built directly on the **past-paper analysis** and **High-Yield Priority List** and follow the slide order of your syllabus deck. Every topic has a **priority tag**, a **simple explanation**, an **analogy**, an **example or pseudocode**, and a **figure**. Diagrams are drawn in plain-text style so they survive copy-paste and printing.

Priority tags used in these notes

Tag	Meaning
[🔥 HIGH-YIELD: Frequently Asked]	Asked in 3 or more of the 5 past papers, or in the Top-5 priority list. Study first.
[⭐ ESSENTIAL FOR EXAM]	Asked in 1 to 2 papers (often the latest ones) or a quick-win topic. Study second.
[📘 GOOD TO KNOW]	Asked once; easy marks if you have a minute.
[💡 WILDCARD: Not asked yet, but in your deck]	Never asked, but the deck has dedicated slides. Skim at the end.

Recommended study order (from the priority list)

Rank	Topic	Where in these notes	Time share
1	Deadlock (conditions, prevention, Banker's, RAG, recovery)	Unit 6	25%
2	Semaphores, producer/consumer, readers/writers, mutual exclusion	Unit 5	22%
3	Threads (thread vs process, ULT vs KLT)	Unit 4	15%
4	Process model: states, suspension, process image / PCB	Unit 3	12%
5	OS fundamentals, kernel, microkernel	Unit 2	10%
Quick wins	SMP/multicore, interrupts, instruction numerical, DMA, cache	Unit 1	16%

Contents

- **Unit 1** - Computer System Overview: components, instruction cycle, numericals, interrupts, memory hierarchy, cache, I/O and DMA, SMP and multicore

- **Unit 2** - Operating System Overview: definition, objectives, services, evolution, kernel, monolithic vs microkernel, storage management
- **Unit 3** - Process Description and Control: process vs program, 5/7-state models, suspension, process image, PCB, process switch
- **Unit 4** - Threads, SMP and Microkernels: thread concept, multithreading, ULT vs KLT, Flynn's classification
- **Unit 5** - Concurrency: race condition, process interaction, mutual exclusion, semaphores, producer/consumer, readers/writers, monitors, message passing
- **Unit 6** - Deadlock and Starvation: conditions, RAG, prevention, Banker's algorithm, detection, recovery, dining philosophers
- **Final Revision Section** - diagrams list, formula sheet, comparison tables, practice questions, answer-writing strategy

How to read the notes

- **Blue boxes** = key points. **Yellow boxes** = real-world analogies. **Green boxes** = worked examples.
- **Red boxes** = exam answer templates (what to write for full marks) or common mistakes.
- Grey code blocks = pseudocode and diagrams. Copy them into your exam answers as the model.
- Past-paper codes: J22 = Jan 2022, F22 = Feb 2022, J23 = Jan 2023, M24 = May 2024, J25 = Jan 2025.

UNIT 1: Computer System Overview (Chapter 1)

Before you can understand an **operating system**, you need a picture of the machine it runs on. This unit covers the hardware ideas that every later unit builds on: the main components, how an instruction executes, interrupts, memory and cache, I/O, and multiprocessors.

🔗 Unit map and priority

Interrupts [★] - asked in M24 and J25 (the two latest papers).

Instruction-format numerical [★] - asked in J23 and J25 (nearly the same question).

Multiprocessor / multicore / SMP [🔥] - asked in J22, F22, J23 (3 of 5 papers).

Cache, DMA / I/O techniques [★] - asked once each (J25, J22), 6-8 marks each.

Top-level components [🔵] and **memory hierarchy numericals** [💡] - less frequent but easy marks.

1.1 Top-Level Components of a Computer

[🔵] **GOOD TO KNOW** Asked in J23 Q1A (7 marks): "four main elements + block diagram"

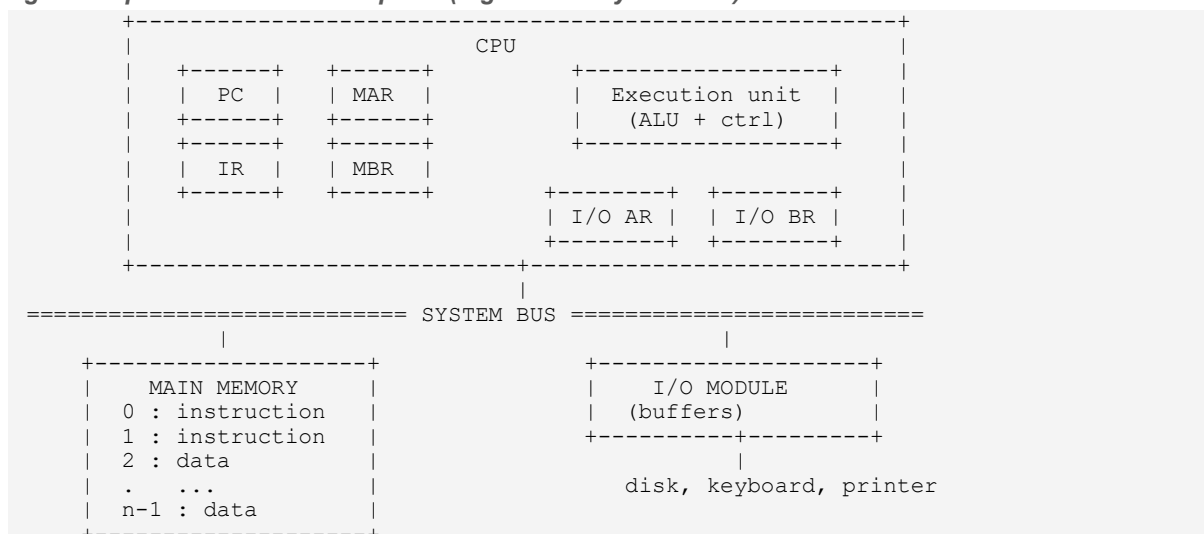
A computer has **four main structural elements**:

- **Processor (CPU)** - the brain. It fetches and executes instructions and does the calculations.
- **Main memory** - the short-term workspace. It stores the running programs and their data. It is volatile (contents vanish when power is off).
- **I/O modules** - the doors to the outside world: keyboard, disk, printer, network card.
- **System bus** - the road network that carries data, addresses and control signals between the other three.

💡 Real-world analogy

Think of a **restaurant kitchen**. The **chef** is the processor. The **counter and shelves** are main memory (ingredients and recipes close at hand). The **delivery door and waiters** are the I/O modules. The **corridors** connecting everything are the system bus.

Figure: Top-level view of a computer (Figure 1.1 in your deck)



Processor registers (the CPU's tiny, super-fast notepad)

Register	What it does
PC (Program Counter)	Holds the address of the next instruction to fetch. Incremented after every fetch.
IR (Instruction Register)	Holds the instruction just fetched (the one being executed).
MAR (Memory Address Register)	Holds the address the CPU wants to read from or write to in memory.
MBR (Memory Buffer Register)	Holds the data being written to memory, or just read from memory.
I/O AR / I/O BR	Same idea as MAR/MBR, but for an I/O device.
PSW (Program Status Word)	Condition codes (positive, zero, overflow), interrupt enable/disable bit, user/supervisor mode.

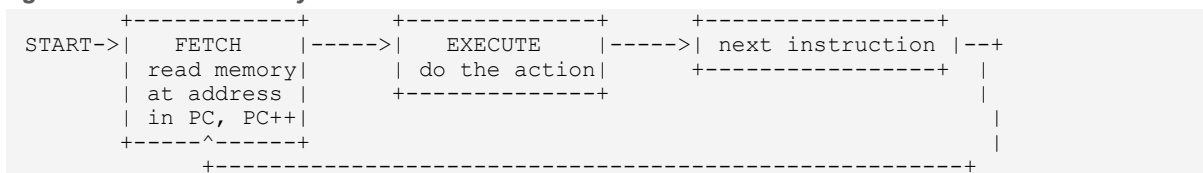
Two groups of registers: **user-visible** (data, address, index, segment pointer, stack pointer - programmers can use them) and **control & status** (PC, IR, PSW - used by the processor and OS).

1.2 Instruction Execution (Fetch-Execute Cycle)

[★ ESSENTIAL FOR EXAM] Foundation for the numerical questions (J23 Q6B, J25 Q6B) and for understanding interrupts

The processor does only two things, forever: **fetch** an instruction from memory, then **execute** it.

Figure: The instruction cycle



After a fetch, the instruction sits in the **IR**. Its bits tell the processor what to do. Actions fall into four categories:

- **Processor-memory** - move data between CPU and memory (LOAD, STORE).
- **Processor-I/O** - send data to or receive data from a device.
- **Data processing** - arithmetic or logic (ADD, AND).
- **Control** - change the order of execution (JUMP, BRANCH).

Example: Hypothetical 16-bit machine (slides 21-25)

Instruction format: **4-bit opcode + 12-bit address**. Opcodes used: 1 = load AC from memory, 2 = store AC to memory, 5 = add memory to AC.

Program (in hex): address 300 holds 1940, 301 holds 5941, 302 holds 2941. Data: location 940 holds 0003, location 941 holds 0002.

Step	What happens	PC after	Registers / memory
1	Fetch 1940 from 300	301	IR = 1940. Opcode 1 = load, address 940, so AC = mem[940] = 0003

Step	What happens	PC after	Registers / memory
2	Fetch 5941 from 301	302	IR = 5941. Opcode 5 = add, address 941, so AC = 0003 + 0002 = 0005
3	Fetch 2941 from 302	303	IR = 2941. Opcode 2 = store, so mem[941] = AC = 0005

Result: the program added the contents of 940 and 941 and stored the sum in 941. Each memory access uses **MAR** (address) and **MBR** (data).

1.3 Instruction-Format Numericals (PC, IR, Memory Size)

[★ **ESSENTIAL FOR EXAM**] Asked in J23 Q6B (4 marks) and J25 Q6B (8 marks) - the SAME question, extended

These questions are pure arithmetic once you know **three rules**:

Three rules

- **Address bits** = instruction length - opcode bits.
- **Addressable memory** = $2^{\text{address bits}}$ locations (bytes if byte-addressable).
- **PC width** = address bits (it stores an address). **IR width** = full instruction length (it stores a whole instruction).

Example: The exact exam question (J23 / J25)

A 32-bit microprocessor has 32-bit instructions with two fields. The **first byte** is the opcode; the **rest** is an immediate operand or an operand address.

(a) Maximum directly addressable memory? Opcode = 8 bits, so address = $32 - 8 = 24$ bits.
Memory = $2^{24} = 16,777,216$ bytes = **16 MB**.

(c) Bits in PC and IR? PC = **24 bits** (holds a 24-bit address). IR = **32 bits** (holds the entire instruction).

(b) Impact of bus width on speed (J25 Q6B-b). The data bus decides how many trips are needed to move a 32-bit instruction; the address bus decides how many trips are needed to send the address:

Bus design	Address cycles	Data cycles	Total and comment
32-bit address bus, 16-bit data bus	1	2	3 bus cycles per instruction/operand. Whole address sent at once; data needs two trips.
16-bit address bus , 16-bit data bus	2	2	4 bus cycles . Address sent in two halves (needs extra latching logic in memory interface) AND data in two halves, so it is slower .

Real-world analogy

A narrow road (16-bit bus) means a 32-bit lorry must make two trips. If the **address** road is also narrow, even the delivery address has to be given in two phone calls. More trips = slower system.

Example: Practice problem (try before reading the answer)

A machine has 16-bit instructions with a 5-bit opcode. How many bits in PC and IR, and how many locations can be addressed?

Answer: address = 16 - 5 = 11 bits, so PC = 11 bits, IR = 16 bits, locations = $2^{11} = 2,048$.

1.4 Interrupts

[★ **ESSENTIAL FOR EXAM**] Asked in M24 Q1A (8 marks: simple + multiple interrupts) and J25 Q5B (6 marks: why required + types)

What is an interrupt and why do we need it?

An **interrupt** is a signal from hardware (or software) telling the processor: "Stop what you are doing, something needs attention." The processor pauses the current program, runs a small routine called the **interrupt handler**, then resumes exactly where it left off.

Why required? The processor is thousands of times faster than I/O devices. Without interrupts the processor would sit idle waiting for a printer or disk. Interrupts let it do useful work while the slow device works, improving **processor utilization**.

Real-world analogy

Without interrupts (polling): you put bread in the toaster and stare at it until it pops, checking every second.

With interrupts: you go and read the newspaper; the toaster **pings** when done. You walk over, butter the toast, and return to the newspaper at the same line.

Types (classes) of interrupts

Type	Cause and example
Program	Raised by an instruction result: arithmetic overflow, divide by zero, illegal instruction, accessing memory outside the user's allowed area.
Timer	Generated by the processor's clock so the OS can regularly regain control (used for time slicing).
I/O	Generated by an I/O controller: operation finished normally, or an error occurred (printer finished a page).
Hardware failure	Power failure, memory parity error.

Program flow with and without interrupts

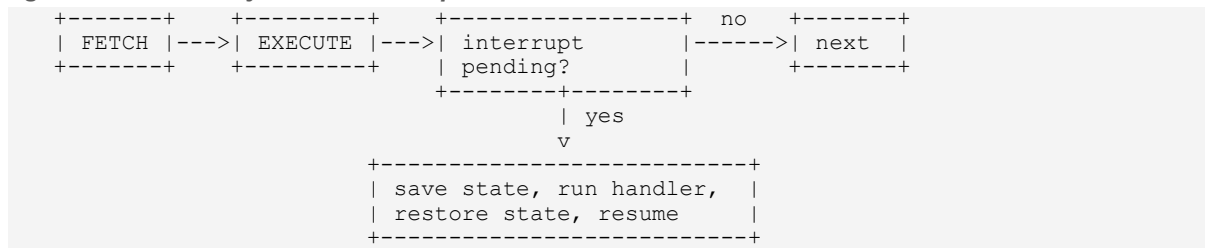
Figure: A program writing to a device (long I/O wait)

WITHOUT INTERRUPTS	WITH INTERRUPTS
-----	-----
1. prepare data	1. prepare data
2. issue I/O command	2. issue I/O command
3. WAIT (poll status...) <-- idle!	3. run other useful code
... ..	... device works in background
4. finish operation	>>> INTERRUPT arrives <<<
	4. handler finishes operation
	5. resume the program

The interrupt cycle

At the end of every instruction the processor **checks for a pending interrupt**. If none, it fetches the next instruction. If one is pending, it suspends the program and runs the handler.

Figure: Instruction cycle with interrupt check



Simple interrupt processing (step by step)

- 1. Device controller issues an interrupt signal.
- 2. Processor **finishes the current instruction** (it never stops mid-instruction).
- 3. Processor acknowledges the device.
- 4. Processor **saves context**: pushes the **PSW and PC** onto the control stack.
- 5. Processor loads the PC with the **address of the interrupt handler**.
- 6. Handler saves the rest of the register state, services the device, then **restores** it.
- 7. Processor pops the old **PSW and PC**: the interrupted program continues as if nothing happened.

Code / pseudocode: interrupt handling in pseudocode

```

on_interrupt(device):
    push(PSW); push(PC)           // save where we were
    PC = handler_address[device] // jump to the handler
    save_registers()
    service(device)               // read data, clear the signal
    restore_registers()
    PC = pop(); PSW = pop()       // resume the interrupted program
  
```

Multiple interrupts (M24 Q1A)

What if a second interrupt arrives while the first is being handled? There are **two approaches**:

	1. Sequential (disable interrupts)	2. Nested (priorities)
Idea	While one interrupt is handled, further interrupts are disabled and wait.	Each interrupt has a priority . A higher-priority one can interrupt a lower-priority handler.
Pros	Simple. Interrupts handled strictly in order of arrival.	Urgent devices get served quickly.
Cons	Ignores urgency: a critical device may wait behind a slow one.	More complex (needs a stack of saved contexts).

🔧 Example: Printer (priority 2), Disk (priority 4), Communication line (priority 5)

A user program runs from $t = 0$. Printer interrupt at $t = 10$, communication at $t = 15$, disk at $t = 20$. Assume each handler needs 10 time units.

Figure: Timeline: sequential vs nested

```

SEQUENTIAL (interrupts disabled during a handler)
time:  0      10      20      30      40
      |--USER---|---PRINTER---|---COMM---|---DISK---|---USER-->
                      (15: comm waits, 20: disk waits)

NESTED (higher priority preempts)
time:  0      10      15      25      35      40
      |--USER---|PRN-|---COMM---|---DISK---|PRN-|---USER-->
                      ^ comm (5) > printer (2): preempts
                        ^ disk (4) arrives at 20, waits for comm,
                          then runs BEFORE printer resumes (4 > 2)

```

Exam answer template: Interrupts (6-8 marks)

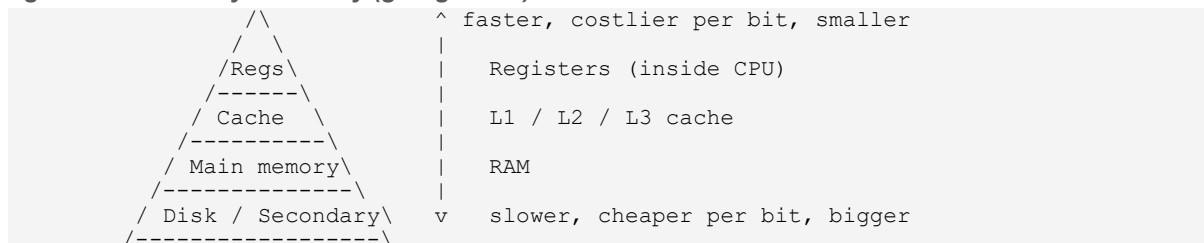
- Define interrupt + why required (processor faster than I/O, better utilization).
- List the 4 types: program, timer, I/O, hardware failure with one example each.
- Draw the interrupt cycle diagram and give the 7 simple-processing steps.
- Explain multiple interrupts: sequential vs nested, with the timeline example above.

1.5 Memory Hierarchy and Locality of Reference

 **WILDCARD: Not asked yet, but in your deck** Not asked yet, but slides 44-52 and Practice Exercise 2 (slide 78) are numericals

Designers face three questions about memory: **How much? How fast? How expensive?** You cannot have all three, so computers use a **hierarchy**: a small amount of very fast memory near the CPU and larger, slower memory further away.

Figure: The memory hierarchy (going down)



Going **down**: cost per bit decreases, capacity increases, access time increases, and the processor accesses it **less frequently**. The last point works because of **locality of reference**.

Locality	Meaning	Example
Temporal	A location used now will probably be used again soon .	A loop counter <code>i</code> is accessed every iteration, so keep it in a register.
Spatial	Locations near the one just used will probably be used soon.	Walking through an array <code>a[0]</code> , <code>a[1]</code> , <code>a[2]...</code> - fetch the neighbours too.

Real-world analogy

A **study desk** (cache) holds the few books you are using now; the **bookshelf** (RAM) holds more; the **library** (disk) holds everything. You rarely walk to the library because most of the time the book you need is on the desk.

Average access time (slide 48 exercise)

Formula, with hit ratio H , level-1 time T_1 and level-2 time T_2 : **Average time = $H \times T_1 + (1 - H) \times (T_1 + T_2)$**

✎ Example: Two-level memory

Level 1: 0.1 μ s. Level 2: 1 μ s. 95% of accesses hit level 1.

Average = $0.95 \times 0.1 + 0.05 \times (0.1 + 1) = 0.095 + 0.055 = \mathbf{0.15 \mu s}$.

✎ Example: Practice Exercise 2 (slide 78)

$T_c = 100$ ns, $C_c = 0.01$ cents/bit, $T_m = 1200$ ns, $C_m = 0.001$ cents/bit.

(a) Cost of 1 MB main memory: 1 MB = $8 \times 2^{20} = 8,388,608$ bits. Cost = $8,388,608 \times 0.001$ cents = 8,388.6 cents = $\approx \mathbf{\$83.89}$.

(b) Cost of 1 MB using cache technology: $8,388,608 \times 0.01$ cents = 83,886 cents = $\approx \mathbf{\$838.86}$ (10 times costlier, but much faster).

1.6 Cache Memory

[★ **ESSENTIAL FOR EXAM**] Asked in J25 Q2C (6 marks): "cache structure and cache read operation with diagrams"

Cache is a small, fast memory placed between the processor and main memory. It holds **copies of recently used parts of main memory**. Main memory is slow compared to the CPU, so every instruction would wait on memory. The cache removes most of that waiting using locality of reference.

Cache structure

- Main memory is divided into fixed-size **blocks** of K words.
- The cache is divided into **lines**; each line holds one block plus a **tag** (which main-memory block it currently holds).
- The cache has far fewer lines than memory has blocks ($C \ll M$), so lines are reused.

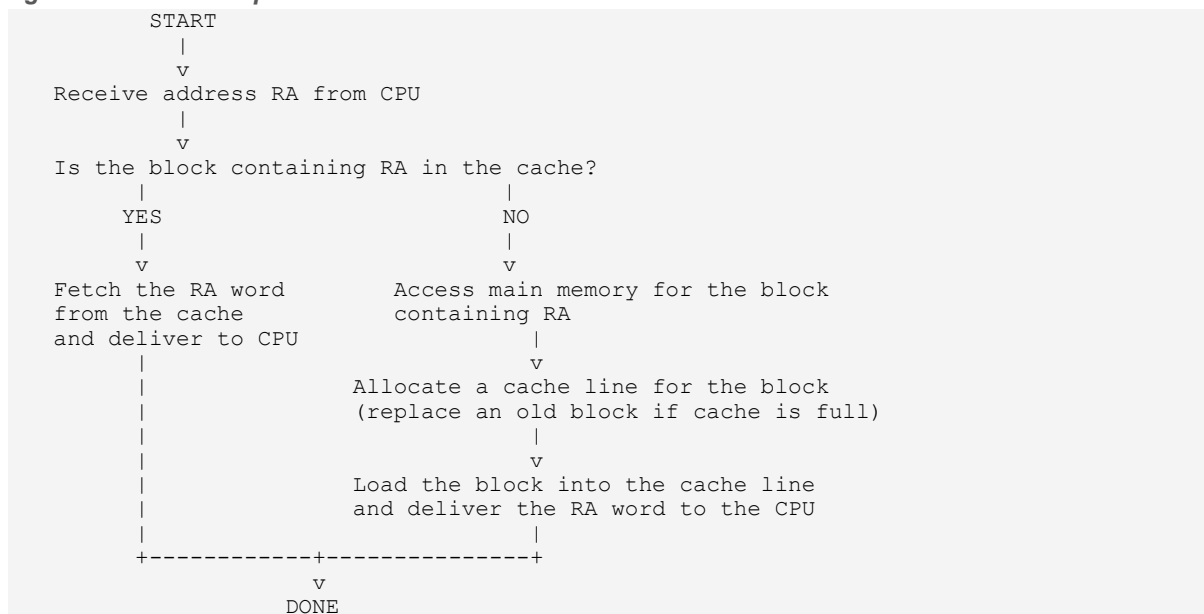
Figure: Cache / main memory structure

CACHE			MAIN MEMORY	
Line	Tag	Block (K words)	Address	Word
0	2	[block 2 data]	0	\
1	7	[block 7 data]	1	> Block 0 (K words)
2	0	[block 0 data]	2	/
...	...	...	3	\ Block 1
C-1	5	[block 5 data]	...	... Block M-1

(only C blocks fit, $C \ll M$)
The TAG tells which of the M blocks is in the line.

Cache read operation (flowchart)

Figure: Cache read operation



Cache design elements

Element	Meaning
Cache size	Even a small cache improves performance a lot.
Block size	Unit moved between memory and cache. Bigger blocks give more hits (spatial locality) up to a point.
Mapping function	Decides which cache line a memory block may occupy.
Replacement algorithm	Which block to evict when full. Common: LRU (Least Recently Used).
Write policy	When to write changed data to memory: every update (write-through) or only when the block is replaced (write-back; saves writes but memory may be stale).

🔧 Example: Hit ratio and speed

Cache access = 10 ns, memory access = 100 ns, 90 of every 100 accesses hit.

Average = $0.9 \times 10 + 0.1 \times (10 + 100) = 9 + 11 = \mathbf{20 \text{ ns}}$ (versus 100 ns with no cache).

Code / pseudocode: why a simple loop is cache-friendly

```
sum = 0
for i = 0 to 999:
    sum = sum + a[i]    // a[i] and a[i+1] are neighbours -> spatial locality
                      // 'sum' and 'i' are reused -> temporal locality
// Result: after the first miss on each block, the next few reads are hits.
```

Disk cache (slide 54) is the same idea in software: part of main memory buffers disk data, so repeated reads avoid the slow disk.

1.7 I/O Techniques and Direct Memory Access (DMA)

[★ **ESSENTIAL FOR EXAM**] Asked in J22 Q7 (8 marks): "list I/O techniques, DMA with block diagram, DMA configurations"

There are **three techniques** for performing I/O:

Technique	How it works	Processor's role	Drawback
Programmed I/O	I/O module does the action and sets bits in a status register. No interrupts.	Keeps checking (polling) the status until done.	Wastes processor time.
Interrupt-driven I/O	Processor issues command, goes on working. Module interrupts when ready.	Saves context, runs handler.	Every word still passes through the processor, so it uses a lot of processor time.
DMA	A DMA module moves a whole block directly between I/O and memory.	Only starts the transfer and gets one interrupt at the end.	Needs extra hardware; competes for the bus.

💡 Real-world analogy

You ordered 1000 pizzas.

Programmed I/O: you stand at the door and carry each pizza into the kitchen yourself.

Interrupt-driven: you do other work, but run to the door for **each** pizza when the bell rings.

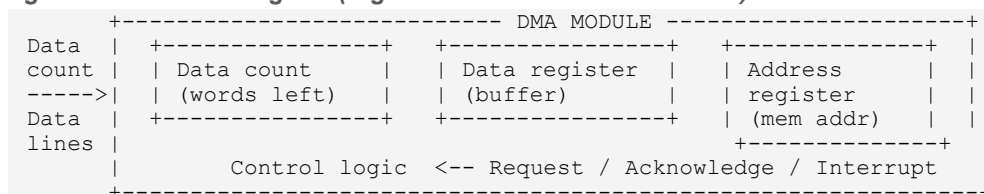
DMA: you hire an assistant who carries all 1000 pizzas into the kitchen and rings the bell **once** when finished.

How DMA works

- 1. The processor tells the DMA module: read or write, the **device address**, the **starting memory address**, and the **number of words**.
- 2. The processor carries on with other work.
- 3. The DMA module transfers the block **directly** to/from memory, one word at a time, without the processor.
- 4. When done, DMA sends **one interrupt** to the processor.

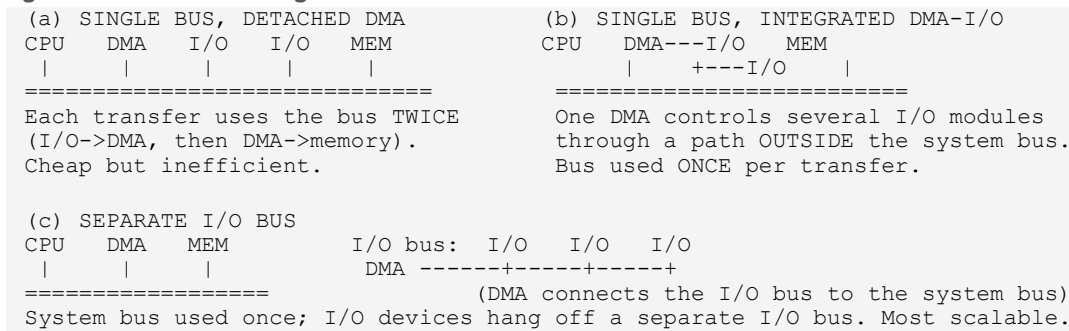
If the DMA and the processor both want the bus, the DMA takes a bus cycle while the processor pauses briefly (**cycle stealing**). The processor is not interrupted; it just waits one cycle.

Figure: DMA block diagram (registers inside the DMA module)



Ways in which DMA can be configured (J22 Q7)

Figure: Three DMA configurations



Exam answer template: I/O techniques and DMA (8 marks)

- List 3 techniques and give a one-line difference for each (use the table).
- Draw the DMA block diagram and explain the 4 steps (processor gives R/W, device address, memory address, count).
- Explain cycle stealing and draw the 3 configurations (a), (b), (c) with the bus-usage point for each.

1.8 Multiprocessor, Multicore and SMP

 **HIGH-YIELD: Frequently Asked** Asked in J22 Q6, F22 Q6 and J23 Q1C (3 of 5 papers, 6-7 marks, near-identical wording)

To go faster, you can **replicate processors**. Three popular ways: **SMP (symmetric multiprocessors)**, **multicore computers**, and **clusters**.

Multiprocessor vs Multicore

	Multiprocessor (SMP)	Multicore (chip multiprocessor)
Definition	A computer with two or more separate processor chips sharing memory and I/O.	Two or more processors (cores) on a single silicon die.
Each unit contains	Full processor with its own cache(s).	Registers, ALU, pipeline, control unit and L1 caches per core; usually a shared L2 (maybe L3) on the chip.
Communication	Over an external bus/interconnect, so slower.	On-chip, so faster and uses less power.
Everyday example	Server with 4 CPU sockets.	Your laptop's quad-core i5/i7.

Real-world analogy

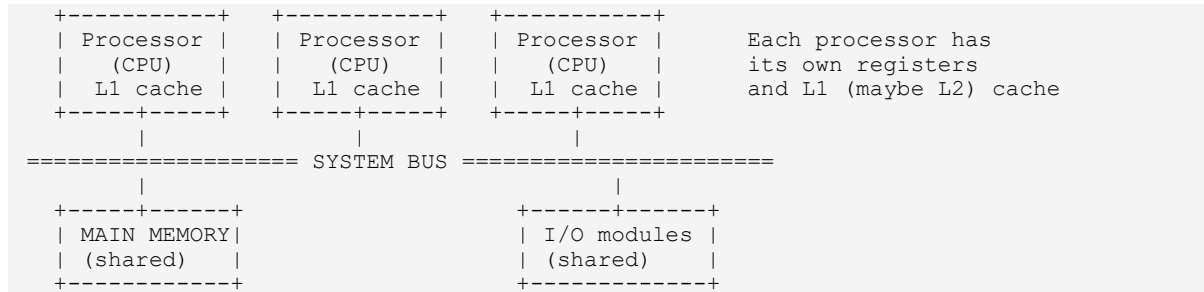
Multiprocessor = a restaurant with several separate kitchens sharing one pantry. **Multicore** = one kitchen with several chefs working side by side.

SMP: five characteristics (slide 69)

- 1. Two or more **similar processors** of comparable capability.
- 2. They **share the same main memory and I/O**, connected by a bus so that memory access time is about the same for every processor.

- **3.** All processors share access to I/O devices.
- **4.** All processors can perform the **same functions** (hence "symmetric").
- **5.** An **integrated operating system** manages the processors and their programs at job, task, file and data-element level.

Figure: SMP organization (Figure for the 6-7 mark answer)



Cache coherence problem

Each processor has its own cache. If CPU-1 changes a word in its cache, CPU-2's cached copy of that word becomes **stale**. Fix: other processors must be told about the update. This is handled in **hardware**, not by the OS.

Advantages of SMP over a uniprocessor

Advantage	Explanation
Performance	If the work can be split, parts run in parallel , so the job finishes faster.
Availability	All processors do the same work, so one failing does not stop the machine ; it continues at reduced performance.
Incremental growth	Need more power? Add another processor .
Scaling	Vendors can sell a range of products (2, 4, 8 CPUs) at different prices from the same architecture.

Exam answer template: Multiprocessor vs multicore, SMP (6-7 marks)

- Two-column table: multiprocessor vs multicore (definition, where cores live, communication).
- 5 characteristics of SMP + the SMP diagram above.
- 4 advantages over a uniprocessor (performance, availability, incremental growth, scaling) with a one-line reason each.
- Mention cache coherence (solved in hardware) to earn the extra mark.

Unit 1 quick revision

- Four elements: processor, memory, I/O, system bus. Registers: PC, IR, MAR, MBR.
- Instruction cycle = fetch (PC to IR, PC++) then execute; interrupt check after each instruction.
- Numerical: address bits = instruction - opcode; PC = address bits; IR = instruction length.
- Interrupts: 4 types; save PSW+PC, run handler, restore; multiple = sequential or nested by priority.
- Cache read: hit gives the word; miss loads the whole block, then gives the word. Average time = $H \times T1 + (1-H)(T1+T2)$.
- I/O: programmed (poll), interrupt-driven, DMA (block transfer, one interrupt). SMP: 5 traits, 4 advantages.

UNIT 2: Operating System Overview (Chapter 2)

This unit answers the most basic question: **what is an operating system and what does it do?** It is the "warm-up" question in every paper, so it is the cheapest place to collect full marks.

Unit map and priority

OS definition, objectives, services, resources, types / evolution [🔥] - asked in J22 Q1, F22 Q1, J25 Q1A (3 of 5 papers).

Kernel, monolithic vs microkernel [🔥] - asked in F22 Q1 and J23 Q1B (7 marks).

Storage-management responsibilities [🟦] - asked in M24 Q1B (8 marks).

Modern OS features, Windows / UNIX / Linux [💡] - never asked, but in the deck.

2.1 What Is an Operating System?

[🔥 **HIGH-YIELD: Frequently Asked**] 5 of 5 papers open with a definitional OS question (J22 Q1, F22 Q1, J25 Q1A and others)

An **operating system (OS)** is a program that **controls the execution of application programs** and acts as an **interface between the user/applications and the computer hardware**. It also **manages the computer's resources**.

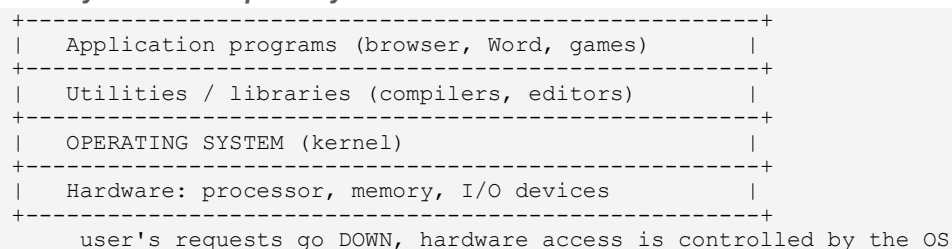
It plays **two roles**:

- **Interface (extended machine)**: hides the ugly hardware details and gives programs simple, uniform services (e.g. "save this file" instead of "move the disk head to track 5...").
- **Resource manager**: shares the processor, memory, disks and devices fairly and efficiently among many programs.

Real-world analogy

The OS is the **manager of a hotel**. Guests (programs) just ask for a room, food or a taxi. The manager (OS) hides the complicated work behind the scenes and decides who gets which room, so that all guests are served fairly and the hotel is used efficiently.

Figure: Layers of a computer system



Examples of operating systems

- **Windows** (Windows 10/11), **Linux** (Ubuntu, Red Hat), **macOS**, **Android**, **iOS**, **UNIX**.

Resources managed by an OS

Resource	What the OS does with it
Processor (CPU)	Decides which process runs when (scheduling).
Main memory	Allocates memory to processes and protects each from the others.

Resource	What the OS does with it
Secondary storage / files	Organizes data into files and directories; controls access.
I/O devices	Provides standard access to keyboards, printers, disks, network.
Information / security	Protects data (confidentiality, integrity, availability).

Objectives of an operating system

Objective	Meaning	Example
Convenience	Makes the computer easier to use.	Drag-and-drop a file instead of typing disk commands.
Efficiency	Uses computer resources in the best possible way.	While one program waits for the printer, the CPU runs another.
Ability to evolve	Allows new features, fixes and hardware without disturbing service.	A security patch or a new USB driver installed without redesigning the OS.

Services provided by an OS

Service	Explanation / example
Program development	Editors, compilers, debuggers (utilities).
Program execution	Loads a program into memory, runs it, ends it.
Access to I/O devices	You call "print"; the OS handles the printer's details.
Controlled access to files	File permissions; who may read/write.
System access	Controls who can use the system and its resources (login).
Error detection and response	Memory errors, device failure, arithmetic overflow, illegal memory access.
Accounting	Collects usage statistics for billing and performance tuning.

Exam answer template: What is an OS? (7-8 marks)

- Definition (controls programs + interface + resource manager) with the hotel analogy.
- Two example OS names (Windows, Linux).
- List of resources managed (processor, memory, files, I/O, security).
- Three objectives with one-line explanations: convenience, efficiency, ability to evolve.
- Services list (program execution, I/O access, file access, error detection, accounting).

2.2 Evolution and Types of Operating Systems

[🔥 HIGH-YIELD: Frequently Asked] J25 Q1A (8 marks): "What is an OS? Explain the types of operating systems."

Operating systems grew step by step as hardware improved. Each stage solved a problem of the one before:

Figure: Evolution of operating systems

Serial Processing --> Simple Batch --> Multiprogrammed Batch --> Time-Sharing
 (no OS, 1 user) (monitor) (several jobs in memory) (many users, interactive)

1. Serial processing (no OS)

- Programmers ran programs directly from a console with **switches and lights**, and a printer.
- Users had to **book machine time**. Setup (loading the compiler, loading source, saving, linking) wasted lots of time.

💡 Real-world analogy

Like a **shared photocopier with a booking sheet**: you book 30 minutes and spend 20 of them setting it up.

2. Simple batch systems

- A program called the **monitor** (the first OS) controls the sequence of jobs. **Similar jobs are batched** together and run one after another.
- When a job ends, it **branches back to the monitor**, which loads the next job automatically.
- **Job Control Language (JCL)** is a special language that gives instructions to the monitor (which compiler, which data).
- Hardware features that were needed: **memory protection** (the user job must not alter the monitor), a **timer** (a job must not hog the CPU), **privileged instructions** (only the monitor may execute some), and **interrupts**.

💡 Real-world analogy

A **laundry service**: the shop collects many customers' clothes, sorts them into batches and runs one load after another with no customer waiting at the machine.

3. Multiprogrammed batch systems

In uniprogramming, when the running job waits for I/O the **processor sits idle**. In **multiprogramming**, several jobs are kept in memory; when one waits for I/O, the processor **switches to another**.

🔧 Example: CPU utilization

A job computes for 1 unit, then waits 4 units for I/O, repeatedly. **Uniprogramming**: CPU busy 1 out of 5 units = **20% utilization**.

Multiprogramming with 3 such jobs interleaved: the CPU can be kept busy about 3 out of 5 units = **~60%**. With 5 jobs it can approach 100%.

Figure: Uniprogramming vs multiprogramming (CPU timeline)

```
UNIPROGRAMMING   : | A-cpu | ....A waits for I/O....| A-cpu | ....  
                  (CPU idle during the dots)  
  
MULTIPROGRAMMING : | A-cpu | B-cpu | C-cpu | A-cpu | B-cpu | C-cpu |  
                  (while A waits for I/O, B and C use the CPU)
```

4. Time-sharing systems

- Uses multiprogramming to handle **multiple interactive users** at once through terminals.
- The processor's time is **sliced and shared** among users; each user feels they have the machine to themselves.
- First example: **CTSS** (Compatible Time-Sharing System) at MIT.

💡 Real-world analogy

A **teacher in a tutorial room** who spends two minutes with each student in turn; every student feels they have a personal teacher.

Comparison table

Type	Key idea	Main problem solved	Example / hardware need
Serial	No OS; manual operation	-	Console, switches
Simple batch	Monitor runs jobs automatically	Setup time wasted	Memory protection, timer, privileged instr.
Multiprogrammed batch	Several jobs in memory	CPU idle during I/O	Interrupts, memory management
Time-sharing	CPU time-sliced across users	Interactive use by many users	CTSS, UNIX

Other types you may also mention (general OS classification, not detailed in your deck): **real-time** OS (hard deadlines, e.g. an airbag controller), **distributed** OS (many computers acting as one), **embedded** OS (small devices), **network** OS.

2.3 Kernel, Monolithic Kernel and Microkernel

[💧 **HIGH-YIELD: Frequently Asked**] F22 Q1 and J23 Q1B (7 marks): "What is a kernel? Differentiate monolithic vs microkernel. Give one example OS of each."

What is the kernel?

The **kernel** (also called the **nucleus**) is the **portion of the OS that is kept in main memory** and contains the **most frequently used functions**. It runs in the privileged (kernel) mode with full access to the hardware.

💡 Real-world analogy

The kernel is the **engine room of a ship**: always running, deep inside, and everything depends on it. Passengers (applications) never go in; they ask the crew (system calls).

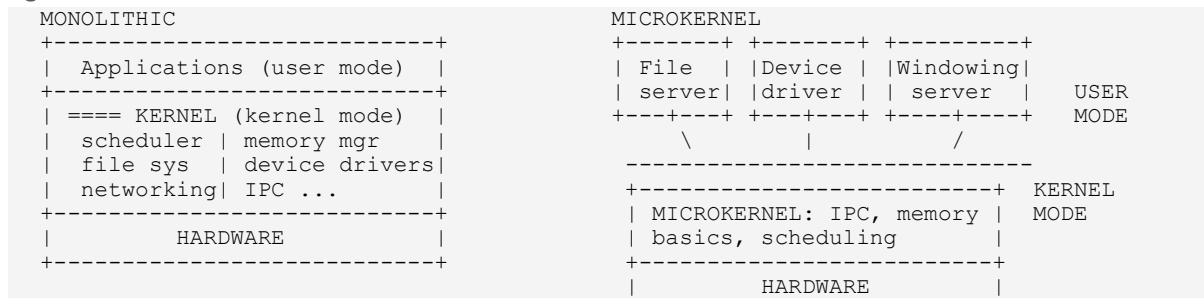
Monolithic kernel

Everything (scheduling, memory management, file system, device drivers, networking) runs together **as one big program in kernel mode**. Parts call each other directly as ordinary function calls, which is **fast**. The drawback: a bug in any part (say, a driver) can **crash the whole system**, and the kernel is huge and hard to maintain.

Microkernel

Only a **few essential functions** stay in the kernel: **address spaces (low-level memory management)**, **inter-process communication (IPC)**, and **basic scheduling**. Device drivers, file system, virtual memory manager, windowing and security services run as **separate user-mode server processes** that communicate through **message passing**.

Figure: Monolithic kernel vs microkernel



Point	Monolithic kernel	Microkernel
Size	Large; contains almost all OS services.	Small; only essential core functions.
Where services run	Inside the kernel, in kernel mode.	Most as user-mode servers.
Communication	Direct function calls (fast).	Message passing through the kernel (slower).
Reliability	A faulty driver can crash the entire OS.	A failing server can be restarted; kernel stays safe.
Extending	Harder; kernel must be rebuilt/reloaded.	Easy; add a new server without touching the kernel.
Performance	Higher.	Lower due to message-passing overhead.
Example OS	Linux, traditional UNIX	QNX, Minix, Mach ; Windows NT uses a modified microkernel

⚠ Common mistake

Your deck never defines "monolithic kernel" but the exam asks it (F22, J23). Use the table above and add class notes. Do not forget the **example OS** - it is a separate mark.

Benefits of the microkernel organization (slides 256-258)

- **Uniform interface** - all services requested the same way, by message passing.
- **Extensibility** and **flexibility** - add or remove services easily.
- **Portability** - to move to a new processor, change only the microkernel.
- **Reliability** - small kernel can be tested rigorously.
- **Distributed system support** - messages can be sent without knowing which machine the target is on.
- **Object-oriented design** - components are objects with defined interfaces.

2.4 Major Achievements of Operating Systems

[GOOD TO KNOW] M24 Q1B (8 marks): "five storage-management responsibilities" comes from here

OS design rests on five big ideas: **processes, memory management, information protection and security, scheduling and resource management, and system structure.**

A. Processes

A **process** is a program in execution. It was invented to solve errors in early multiprogramming systems: improper synchronization, failed mutual exclusion, non-deterministic results and deadlocks (all covered in Units 3, 5, 6). A process has three components: **executable program**, **associated data**, **execution context**.

B. Memory management (storage management)

Users need a computing environment that supports **modular programming** and **flexible use of data**. The OS has **five storage-management responsibilities** (M24 Q1B):

Responsibility	Meaning and example
1. Process isolation	One process must not read or write another's memory. (Your browser tab crash must not corrupt your music player.)
2. Automatic allocation and management	OS gives memory when needed and takes it back; the programmer need not care. (<code>malloc/new</code> just works.)
3. Support of modular programming	Programmers can create, destroy and resize modules (functions/libraries) at run time.
4. Protection and access control	Sharing memory is allowed but controlled (read-only code shared by many users).
5. Long-term storage	Keeps information on disk as files, even after power-off.

The OS meets these needs with **virtual memory** and the **file system**. **Virtual memory** lets programs address memory logically, regardless of how much physical RAM exists. **Paging** divides a process into fixed-size **pages**; a virtual address = page number + offset, mapped dynamically to a physical address in main memory.

💡 Real-world analogy

Virtual memory is like a **huge desk** you imagine you have; the OS secretly keeps only the papers you are currently using on the real small desk and the rest in the cupboard (disk), bringing them over when needed.

C. Information protection and security

Four categories: **availability** (protect against interruption), **confidentiality** (no unauthorized reading), **data integrity** (no unauthorized modification), **authenticity** (verify identities and messages).

D. Scheduling and resource management

Any policy must balance **fairness** (equal access), **differential responsiveness** (urgent job classes get quicker service) and **efficiency** (maximize throughput, minimize response time).

E. System structure

A big OS is organized as a **series of levels (layers)**, each built on the one below, to make it easier to design, test and modify (e.g. level 1 electronic circuits ... level 5 processes ... level 9 files).

2.5 Modern Operating System Features

[💡 **WILDCARD: Not asked yet, but in your deck**] Not asked in 5 papers; deck slides 130-150. Read once for completeness.

Feature	Idea
Microkernel architecture	Only a few essential functions in the kernel (see 2.3).
Multithreading	A process is divided into threads that run concurrently (Unit 4).
Symmetric multiprocessing (SMP)	Several identical processors share memory and I/O (Unit 1 and Unit 4).
Distributed operating systems	Many machines appear to the user as one system.
Object-oriented design	OS components are objects; makes adding extensions easy.

Windows uses a **modified microkernel** design with an *Executive*, a *Hardware Abstraction Layer (HAL)*, and treats processes/threads as **objects**. **UNIX** has hardware surrounded by the OS kernel, with system-call interface to user programs; modern UNIX kernels (e.g. SVR4) and **Linux** are largely monolithic but modular.

Unit 2 quick revision

- OS = program that controls applications + interface to hardware + resource manager.
- Objectives: convenience, efficiency, ability to evolve. Services: program execution, I/O, files, errors, accounting.
- Evolution: serial -> simple batch (monitor, JCL) -> multiprogramming -> time-sharing (CTSS).
- Kernel = always-in-memory core. Monolithic (Linux/UNIX) = all in kernel; microkernel (QNX/Mach/Minix) = IPC + memory + scheduling only.
- Storage responsibilities: isolation, automatic allocation, modular programming, protection/access control, long-term storage.

UNIT 3: Process Description and Control (Chapter 3)

A **process** is the central idea of every operating system. This unit explains what a process is, the **states** it moves through, and the **data structures** the OS keeps about it.

Unit map and priority

Process vs program; 5-state and 7-state models; suspension [🔥] - F22 Q2, M24 Q2A, J25 Q2A (3 of 5 papers).

Process image and PCB [⭐] - J23 Q2A (7 marks).

Process creation/termination, process switch, modes of execution [💡] - not yet asked, but useful for the 7-state model answers.

3.1 What Is a Process? Process vs Program

[🔥 **HIGH-YIELD: Frequently Asked**] F22 Q2 (7 marks): "Is process and program the same? Give your views." Also J22 Q2: "What is a process?"

A **program** is a passive set of instructions stored on disk (like a file). A **process** is a **program in execution**: a living, running instance with a current state and resources.

Formal definitions from your deck: a process is (1) a program in execution, (2) an instance of a program running on a computer, (3) the entity that can be assigned to and executed on a processor, (4) a unit of activity characterized by a **sequence of instructions, a current state, and a set of associated system resources**.

Real-world analogy

A **recipe** in a cookbook is the *program*. **Cooking the dish right now** (with ingredients out, a pot on the stove, a timer running) is the *process*. The same recipe can be cooked by two cooks at once: **one program, two processes**.

Point	Program	Process
Nature	Passive (a file of instructions)	Active (executing)
Lifetime	Permanent, stays on disk	Temporary: created, runs, terminates
Resources	None	Memory, CPU, files, I/O devices
State	No state	Has state (ready, running, blocked...)
Example	chrome.exe on disk	Chrome running with 3 tabs; two windows = two processes

Process elements (what the OS must remember about a process)

Identifier, state, priority, program counter, memory pointers (code, data, shared blocks), **context data** (register contents), **I/O status information** (outstanding I/O requests, devices and files in use), **accounting information** (CPU time used, time limits).

All these are stored in a data structure called the **Process Control Block (PCB)**, created and managed by the OS. The PCB lets the OS **interrupt a process and resume it later** as if nothing had happened, which is what makes multiprocessing possible.

Exam answer template: Process vs program (7 marks)

- Say clearly: **No, they are not the same.** Process = program in execution.
- Give the comparison table (passive vs active, resources, lifetime, state).
- Recipe vs cooking analogy and 'one program, many processes' example. Then lead into the 5-state model.

3.2 Process States and Transitions

[🔥 HIGH-YIELD: Frequently Asked] F22 Q2 (5-state with diagram), M24 Q2A (7-state with diagram, 8 marks), J25 Q2A (reasons for suspension, 6 marks)

A process is not always running. It changes **state** as it executes. We build the model in steps, simplest first.

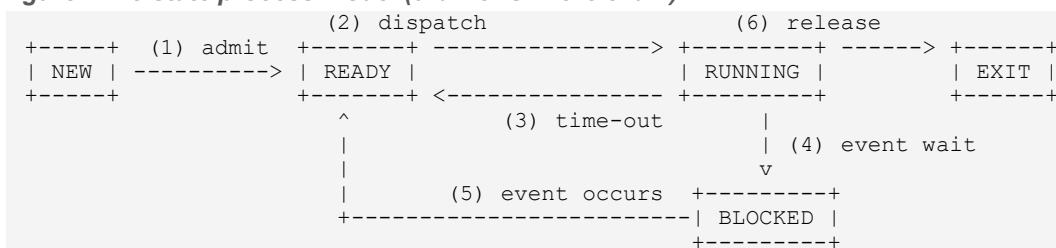
Step 1: Two-state model

A process is either **Running** (on the CPU) or **Not-running** (waiting in a queue). Problem: some not-running processes are ready to run, others are **blocked** waiting for I/O. A simple FIFO queue would wrongly hand the CPU to a blocked process. So we need more states.

Step 2: Five-state model

State	Meaning
New	The OS has created the process (PCB made) but has not yet admitted it to the pool of executable processes.
Ready	Prepared to execute; waiting for the CPU .
Running	Currently executing on the processor (only one per CPU).
Blocked / Waiting	Cannot continue until some event occurs (I/O completion, a signal).
Exit	Halted or aborted; the OS is cleaning up (releasing resources).

Figure: Five-state process model (draw this in the exam)



#	Transition	When and why it happens
1	New to Ready (admit)	OS is ready for another process (enough memory/resources) and admits it.
2	Ready to Running (dispatch)	The scheduler picks this process from the ready queue and gives it the CPU.
3	Running to Ready (time-out)	The process used up its time slice , or a higher-priority process became ready. It is pre-empted.
4	Running to Blocked (event wait)	The process asks for something it must wait for, e.g. a disk read.

#	Transition	Explanation
3-7	Same as five-state model	dispatch, time-out, event wait, event occurs, release.
8	Blocked to Blocked/Suspend	Memory is needed and no ready process is available; swap a blocked process to disk.
9	Blocked/Suspend to Ready/Suspend	The awaited event occurs while the process is still on disk.
10	Ready to Ready/Suspend	OS must free memory and prefers to suspend a low-priority ready process rather than a high-priority blocked one.
11	Ready/Suspend to Ready	No ready process in memory, or a suspended ready process has higher priority; swap it in .
12	Blocked/Suspend to Blocked	Memory has become free and the process (whose event will soon occur) is brought back.
+	Running to Ready/Suspend	A running process is pre-empted by a higher-priority process whose swap-in frees memory, so the OS may suspend the pre-empted one.

Reasons for process suspension (J25 Q2A, 6 marks)

Reason	Explanation
Swapping	OS needs to release enough main memory to bring in a ready process.
Other OS reason	OS suspects the process is causing a problem (e.g. a background or utility process).
Interactive user request	User wants to suspend the program for debugging or in connection with a resource use (Ctrl+Z).
Timing	A periodic process (e.g. system monitor) is suspended while waiting for the next interval.
Parent process request	A parent may suspend a child to examine or modify it, or to coordinate with other children.

Example: Following one process through the model

You press Play in a media player. **New** (process created) -> **Ready** (admitted) -> **Running** (dispatched) -> **Blocked** (waits to read the song from disk) -> **Ready** (read finished) -> **Running** -> ... You minimise the app and memory runs low: **Blocked/Suspend** (swapped to disk) ... You return to it: **Ready/Suspend** -> **Ready** -> **Running** -> **Exit** (you close it).

Exam answer template: 5-state / 7-state model (6-8 marks)

- State each process state in one line. Draw the diagram **with every transition labelled**.
- Explain each transition (use the table). For the 7-state model, add why suspend states are needed (CPU faster than I/O; swapping).
- List the 5 reasons for suspension if asked.

3.3 Process Image and Process Control Block (PCB)

[★ ESSENTIAL FOR EXAM] J23 Q2A (7 marks): "What is a process image? Discuss its elements."

The **process image** is the **complete collection of everything belonging to a process** that the OS keeps in memory. It has **four elements**:

Element	Meaning
Program (code)	The instructions to be executed.
Data	Static and global variables (a user data area).
Stack	Used for procedure calls, parameters and return addresses. At least one stack per process.
Process Control Block (PCB)	All the information the OS needs to manage the process (the OS's file on that process).

Figure: A process image in virtual memory

```

process image (one per process)
+-----+ <- bottom of address space
| Process control block | OS information about the process
+-----+
| User stack           | procedure calls, local variables
| (grows downward)    |
+-----+
| User data            | global / static variables
+-----+
| User program (code)  | instructions
+-----+

```

💡 Real-world analogy

The process image is your **entire student file at the university**: the course syllabus (program), your notebooks (data), a to-do pile that grows and shrinks (stack), and the **registration card** the office keeps about you (PCB).

Contents of the PCB (three categories)

Category	Typical items
1. Process identification	Process ID (PID), parent process ID, user ID.
2. Processor state information	User-visible registers ; control and status registers (PC, condition codes, status/PSW); stack pointers .
3. Process control information	Scheduling and state information (state, priority, scheduling info, awaited event); data structuring (links to other processes: queue, parent/child); inter-process communication flags and messages; process privileges ; memory management (segment/page table pointers); resource ownership and utilization (open files, CPU used).

Why is the PCB so important? It is the **only record** the OS has of the process. When a process is switched out, the OS **saves the registers into the PCB**; when it is switched back in, the OS **restores them**. The PCB is the **set of data that allows the OS to control the process**.

Code / pseudocode: a PCB as a C structure (simplified)

```
struct PCB {
    int    pid;           // identification
    int    parent_pid;
    int    state;         // READY, RUNNING, BLOCKED ...
    int    priority;      // scheduling info
    int    program_counter; // processor state
    int    registers[16];
    void *page_table;     // memory management info
    int    open_files[20]; // resource info
    struct PCB *next;     // link into a ready/blocked queue
};
```

3.4 Process Creation, Termination and Switching

[💡 **WILDCARD: Not asked yet, but in your deck**] Not asked in 5 papers; deck slides 165-167, 197-202

Process creation (five steps, slide 198)

- **1. Assign a unique process identifier.**
- **2. Allocate space** for the process image.
- **3. Initialise the PCB.**
- **4. Set up linkages** (e.g. add to the scheduling queue).
- **5. Create or expand other data structures** (e.g. accounting file).

Typical reasons for creation: a **new batch job**, an **interactive logon**, the OS creates it to provide a service (e.g. printing), or an existing process **spawns** a new one.

Termination reasons: normal completion, time limit exceeded, memory unavailable, protection error, arithmetic error, I/O failure, invalid instruction, privileged instruction, operator/OS intervention, parent termination, parent request.

Modes of execution

- **User mode** - less privileged; user programs run here.
- **Kernel (system / control) mode** - more privileged; the OS kernel runs here and can execute privileged instructions and touch protected memory.

When does the OS switch a process? (slides 199-200)

Event	Meaning
Clock interrupt	Process used its maximum time slice.
I/O interrupt	An I/O operation completed; the waiting process may now run.
Memory fault	Needed address is in virtual memory; must be brought in first.
Trap	An error or exception; may move the process to Exit.
Supervisor call	Process asks the OS for a service, e.g. file open.

Steps in a process switch (slides 201-202)

- **1. Save the processor context** (PC and registers). **2. Update the PCB** of the running process (new state, etc.).
- **3. Move its PCB** to the right queue (ready, blocked, ready/suspend). **4. Select another process.**

- **5.** Update the PCB of the selected process to Running. **6.** Update memory-management structures. **7.** Restore the selected process's context.

Common mistake

A **mode switch** (user to kernel on an interrupt) is **cheaper** and does not necessarily change the running process. A **process switch** changes which process is running and needs all 7 steps above.

Unit 3 quick revision

- Program = passive; process = program in execution with state and resources.
- 5-state: New, Ready, Running, Blocked, Exit. Transitions: admit, dispatch, time-out, event wait, event occurs, release.
- 7-state adds Blocked/Suspend and Ready/Suspend because CPU is faster than I/O (swapping).
- Suspension reasons: swapping, other OS reason, interactive user, timing, parent request.
- Process image = program + data + stack + PCB. PCB = identification + processor state + process control information.

UNIT 4: Threads, SMP and Microkernels (Chapter 4)

Processes are heavy. This unit introduces **threads**, a lighter way to do several things at once inside one program, and how they relate to multiprocessors and microkernels.

🔗 Unit map and priority

Thread concept, thread vs process, single vs multithreaded, multithreading vs multiprocessing [🔥] - J22 Q3, J23 Q2B, M24 Q2B.

User-level vs kernel-level threads [🔥] - J22 Q3, J23 Q2C, J25 Q5A (3 times, almost word for word).

Parallel architectures (SISD/SIMD/MISD/MIMD), SMP OS design [💡] and **microkernel** (see Unit 2, [🔥]).

4.1 What Is a Thread?

[🔥 **HIGH-YIELD: Frequently Asked**] J22 Q3, J23 Q2B (7 marks), M24 Q2B (8 marks): thread concept, thread vs process, multithreading vs multiprocessing

A traditional process does **two jobs**: (1) **owns resources** (address space, files, I/O devices) and (2) **is scheduled and executed** (a path of execution). Modern OSs separate these two jobs:

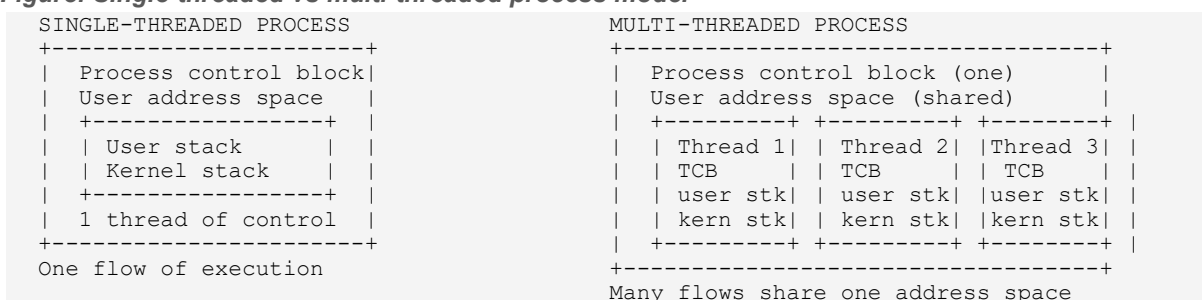
- The unit of **resource ownership** is called the **process (or task)**.
- The unit of **dispatching** (execution) is called the **thread (or lightweight process)**.

Definition: a **thread** is a **dispatchable unit of work inside a process**. It has its own execution state, saved context, **execution stack** and some private storage for local variables, but it **shares the memory, files and resources of its process** with the other threads.

💡 Real-world analogy

A process is a **house** (walls, kitchen, electricity are shared resources). Threads are the **people living in it**: each person does their own task (one cooks, one studies, one watches TV) at the same time but all use the same house. Building a new house (process) is expensive; one more person moving in (thread) is cheap.

Figure: Single-threaded vs multi-threaded process model



Process vs thread

Point	Process	Thread
Weight	Heavy-weight	Light-weight
Resources	Owns address space, files, devices	Shares its process's resources; has only own stack, registers, state

Point	Process	Thread
Creation/termination	Slow (allocate memory, PCB...)	Fast
Context switch	Slow	Fast (same address space)
Communication	Needs kernel (IPC)	Directly through shared memory, without kernel
Protection	Processes are isolated from each other	Threads are not isolated; one bad thread can corrupt the others

Benefits of threads (slide 219)

- Takes **less time to create** a thread than a process; **less time to terminate** one.
- **Less time to switch** between two threads of the same process.
- Threads of a process share memory and files, so they can **communicate without calling the kernel**.

Uses of threads

- **Foreground and background work** (a spreadsheet: one thread reads input, another recalculates).
- **Asynchronous processing** (a word processor's thread auto-saves every minute).
- **Speed of execution** (compute one batch while reading the next).
- **Modular program structure** (a server with a thread per client).

Thread states and operations

States: **Running, Ready, Blocked**. Operations: **Spawn** (create another thread), **Block, Unblock, Finish** (deallocate register context and stacks). **Suspending a process suspends all its threads** (same address space); **terminating a process terminates all its threads**.

Code / pseudocode: multithreading in a small program (pseudocode)

```
main():
    t1 = spawn(download_file) // one process
    t2 = spawn(show_progress) // thread 1: network I/O
    t3 = spawn(save_to_disk)   // thread 2: updates the screen
    join(t1, t2, t3)           // thread 3: disk I/O
    // wait until all finish
// While t1 waits for the network, t2 and t3 keep running.
```

Multithreading vs multiprocessing (M24 Q2B)

	Multithreading	Multiprocessing
Meaning	Many threads within one process (software concept: OS supports multiple threads of execution in a single process).	Using two or more processors (CPUs) to run processes/threads (hardware concept).
Level	Software / OS design.	Hardware / system organisation.
Focus	Concurrency inside a program.	True parallel execution on several CPUs.
Can they coexist?	Yes: threads of one process can run on different processors at the same time in an SMP system. They are not the same thing.	

One-line answer to "Is it the same as multiprocessing?": No. Multithreading is multiple threads in one process (can run even on a single CPU by time-slicing); multiprocessing is multiple processors executing simultaneously.

Exam answer template: Thread (7-8 marks)

- Define thread (dispatchable unit; own stack/registers; shares process resources).
- Draw single-threaded vs multi-threaded model.
- Table: thread vs process. Benefits (create, terminate, switch faster; communicate without kernel).
- Multithreading vs multiprocessing with the 'not the same' conclusion.

4.2 User-Level Threads (ULT) vs Kernel-Level Threads (KLT)

[🔥 HIGH-YIELD: Frequently Asked] Asked three times: J22 Q3, J23 Q2C (6 marks), J25 Q5A (6 marks)

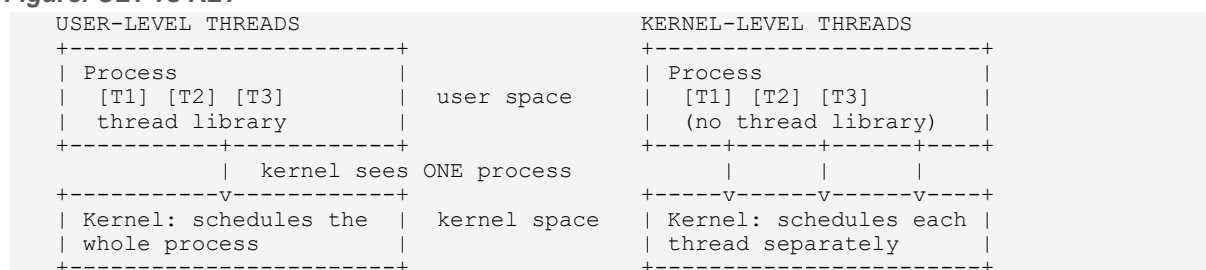
User-level threads (ULT)

All thread management is done by the application using a **thread library** (create, schedule, switch, destroy). The **kernel is unaware** that threads exist; it schedules the **process** as a single unit.

Kernel-level threads (KLT)

Thread management is done by the kernel. There is **no thread library** in the application; it just uses the kernel's thread API. The kernel keeps context for the process **and each thread**, and **scheduling is done on a thread basis**. Example: **Windows**.

Figure: ULT vs KLT



Point	ULT	KLT
Managed by	Application (thread library)	Kernel
Kernel awareness	Kernel does not know of threads	Kernel knows each thread
Thread switch	No mode switch, so fast	Needs a mode switch into the kernel, so slower
Blocking system call	Blocks the entire process (all its threads)	Only that thread blocks; kernel runs another thread of the same process
Multiprocessor	Cannot use several CPUs for one process	Can schedule threads of one process on several CPUs at once
Scheduling	Application-specific (can be tailored)	Kernel's general policy
Portability	Runs on any OS (even without thread support)	Needs OS thread support

Point	ULT	KLT
Example	Early Java green threads, POSIX user libraries	Windows, Linux (NPTL)

Advantages and disadvantages (what examiners want listed)

	Advantages	Disadvantages
ULT	Saves the overhead of two mode switches (user to kernel and back). Scheduling can be application specific . Can run on any OS .	When one thread makes a blocking system call , all threads in the process block . A multithreaded application cannot take advantage of multiprocessing .
KLT	Kernel can schedule multiple threads of the same process on multiple processors simultaneously. If one thread blocks, the kernel can schedule another thread of the same process . Kernel routines can themselves be multithreaded .	A mode switch to the kernel is needed to transfer control from one thread to another within the same process, which is slower.

Remedy for the ULT blocking problem: jacketing. Convert a blocking system call into a **non-blocking** one: instead of calling the system I/O routine directly, the thread calls an application-level **I/O jacket routine** that first checks whether the device is busy and, if so, lets another thread run.

Combined approach (e.g. Solaris)

Thread **creation** is done in user space, and the **bulk of scheduling and synchronisation** of threads within an application is also done in user space; the programmer adjusts how many kernel threads the user threads are mapped onto. This gets **the benefits of both**.

Exam answer template: ULT vs KLT (6-7 marks)

- One-line definition of each (who manages threads; kernel awareness).
- Two-column difference table (at least 6 rows).
- List advantages and disadvantages of each, exactly as in the table. Mention jacketing and the combined approach (Solaris) for bonus.

4.3 Parallel Architectures and SMP Operating Systems

 **WILDCARD: Not asked yet, but in your deck** Flynn's classification and SMP OS design are on slides 241-253; not asked yet

Flynn's classification of parallel processing

Type	Meaning	Example / use
SISD	Single Instruction, Single Data stream. One processor executes one instruction stream on data in one memory.	Traditional single-core von Neumann computer.

Type	Meaning	Example / use
SIMD	Single instruction, Multiple data. One control unit; many processing units run the same instruction on different data .	Vector/array processors, GPUs (add 1 to every pixel).
MISD	Multiple instructions, Single data. Several units run different instructions on the same data. Never commercially implemented ; used in fault-tolerant redundant systems.	Flight-control redundancy.
MIMD	Multiple instructions, Multiple data. Processors run different instruction streams on different data, asynchronously.	SMP servers, clusters, multicore PCs.

SMP operating system

In **symmetric multiprocessing** the **kernel can execute on any processor**, and typically **each processor does its own scheduling** from the pool of available processes or threads. Design issues for a multiprocessor OS:

- **Simultaneous concurrent processes or threads** - kernel routines must be re-entrant.
- **Scheduling** - any processor may do it, so conflicts must be avoided.
- **Synchronisation** - mutual exclusion and ordering of events on shared resources.
- **Memory management** - same issues as uniprocessor plus coordinating paging and **cache coherence**.
- **Reliability and fault tolerance** - graceful degradation if a processor fails.

(See Unit 1, section 1.8 for the SMP diagram, characteristics and advantages. See Unit 2, section 2.3 for the microkernel design, benefits and the monolithic comparison.)

Unit 4 quick revision

- Thread = unit of dispatching; process = unit of resource ownership. Threads share code, data, files; each has its own stack, registers, state.
- Thread vs process: lighter, faster to create/switch, communicate without kernel, but no mutual protection.
- Multithreading (many threads in a process) is NOT multiprocessing (many CPUs).
- ULT: app-managed, fast switch, any OS, but one blocking call blocks all and no multi-CPU. KLT: kernel-managed, multi-CPU and per-thread blocking, but mode switch cost.
- Flynn: SISD, SIMD, MISD (never built), MIMD. SMP kernel can run on any processor.

UNIT 5: Concurrency - Mutual Exclusion and Synchronization (Chapter 5)

When several processes run "at the same time" and share things, strange bugs appear. This unit explains **why**, and the tools (**semaphores**, etc.) that fix them. It carries the most pseudocode questions in the exam.

🔗 Unit map and priority

Semaphores + Producer/Consumer [🔥] - F22 Q3, J23 Q4C, J25 Q4A (and the classic readers/writers in M24).

Readers/Writers with writers' priority [★] - M24 Q3A (8 marks).

Process interaction (3 degrees of awareness) [★] - J23 Q3 (10 marks). **Mutual exclusion, critical section** [★] - J22 Q2.

Hardware support, monitors, message passing [💡] - never asked, but in the deck (slides 294-342).

5.1 Why Concurrency Is Hard: The Race Condition

[★ ESSENTIAL FOR EXAM] Foundation for every topic in this unit

Concurrency means several processes or threads making progress during the same period (by interleaving on one CPU or running in parallel on many). It appears in **multiple applications** (multiprogramming), **structured applications** (one app = many processes) and in **OS structure** (the OS itself is a set of processes/threads).

The root problem: **the relative speed of processes is unpredictable**, and they **share global resources**. Errors become hard to reproduce.

🔗 Example: The echo() example from your deck (slides 282-284)

`echo()` reads a character into the **shared** global `chin`, copies it to `chout` and prints it. Two processes P1 and P2 call it (single processor).

Code / pseudocode: shared procedure

```
char chin, chout;           // SHARED global variables
void echo() {
    chin = getchar();        // (1) read a char into chin
    chout = chin;            // (2) copy it
    putchar(chout);          // (3) print it
}
```

Figure: Interleaving that goes wrong

```

P1                                P2
chin = getchar(); // reads 'x'
---- interrupt, switch to P2 ---->
                                chin = getchar(); // reads 'y' (x is LOST)
                                chout = chin;
                                putchar(chout);    // prints 'y'

<---- switch back to P1 -----
chout = chin; // chin is now 'y'
putchar(chout); // prints 'y' AGAIN. 'x' was never printed!
```

Fix: only one process at a time may be inside `echo()`. That rule is called **mutual exclusion**. The next question: **how do we enforce it?**

💡 Real-world analogy

A **single-person toilet** with a lock: whoever is inside must finish before the next person enters. If two people walk in together, there is a mess. The lock = mutual exclusion.

What the OS must worry about

- Keep track of processes (PCBs); allocate/deallocate resources (CPU, memory, files, I/O devices).
- **Protect** each process's data and resources.
- Make the **output independent of relative speeds** of the processes (speed independence).

5.2 Process Interaction: Three Degrees of Awareness

[★ **ESSENTIAL FOR EXAM**] J23 Q3 (10 marks): "List three degrees of awareness and define each: relationship, influence, potential problems"

To understand how to get speed independence, look at **how processes interact**. There are three degrees of awareness:

Degree of awareness	Relationship	Influence one process has on the other	Potential control problems
1. Processes unaware of each other	Competition for shared resources (independent processes that happen to need the same device).	Results of one process are independent of the other. Timing of the process may be affected.	Mutual exclusion, deadlock (renewable resources), starvation .
2. Processes indirectly aware of each other	Cooperation by sharing (they share objects such as a file or database, without knowing each other's identity).	Results of one may depend on information obtained from others. Timing may be affected.	Mutual exclusion, deadlock, starvation, plus data coherence .
3. Processes directly aware of each other	Cooperation by communication (they know each other by process ID and exchange messages).	Results of one may depend on information from others. Timing may be affected.	Deadlock (consumable resources, e.g. blocked on receive), starvation .

🔑 Example: One example per degree

Unaware: a text editor and a music player both want the sound card or printer (competition).

Indirectly aware: two bank-teller processes updating the same account balance in a shared database (cooperation by sharing).

Directly aware: a web browser process sending a request message to a download-manager process and waiting for the reply (cooperation by communication).

Competition among processes: three control problems

- **Mutual exclusion** - only one process at a time in its **critical section** (e.g. only one process at a time sends commands to the printer).

- Enforcing mutual exclusion creates two more problems: **deadlock** (P1 holds R1 and wants R2, P2 holds R2 and wants R1) and **starvation** (a process is perpetually denied access while others proceed).

5.3 Critical Section and Mutual Exclusion

[★ ESSENTIAL FOR EXAM] J22 Q2 (7 marks): "explain mutual exclusion and critical section"

Critical section: the part of a process's code that **accesses a shared resource** (shared variable, file, printer) and **must not be executed by more than one process at a time**.

Mutual exclusion: the rule that **if one process is in its critical section, no other process may be in its critical section** for the same resource.

Code / pseudocode: structure of every concurrent process

```
while (true) {
    /* entry section: ask permission (enforce mutual exclusion) */
    /* CRITICAL SECTION: touch the shared resource */
    /* exit section: release permission */
    /* remainder section: everything else */
}
```

Requirements for mutual exclusion (slides 291-292)

- 1. Mutual exclusion must be **enforced**: only one process at a time in the critical section for a resource.
- 2. A process that **halts in its non-critical section** must do so **without interfering** with others.
- 3. **No deadlock or starvation** allowed.
- 4. A process **must not be delayed** from entering the critical section when no other process is using it.
- 5. **No assumptions** about relative process speeds or number of processors.
- 6. A process stays inside its critical section for a **finite time** only.

Three ways to achieve mutual exclusion (slide 293)

Approach	Details
1. Hardware support	Interrupt disabling; special machine instructions (compare-and-swap, exchange).
2. OS / language support	Semaphores , monitors, message passing (most important for exams).
3. Software by the processes themselves	Processes coordinate among themselves with no OS or language help (e.g. Dekker's and Peterson's algorithms).

Hardware support (not asked yet; know the basics)

Code / pseudocode: interrupt disabling (uniprocessor only)

```
while (true) {
    /* disable interrupts */
    /* critical section */
    /* enable interrupts */
    /* remainder */
}
```

- Advantage:** simple. **Disadvantages:** the CPU cannot interleave programs (efficiency drops); **does not work on multiprocessors** (disabling interrupts on one CPU does not stop another).

Code / pseudocode: compare_and_swap and exchange (each executes ATOMICALLY, i.e. cannot be interrupted)

```
int compare_and_swap(int *word, int testval, int newval) {
    int oldval = *word;
    if (oldval == testval) *word = newval;
    return oldval;
}

void exchange(int *register, int *memory) {
    int temp = *memory; *memory = *register; *register = temp;
}

/* mutual exclusion with compare_and_swap; bolt is shared, starts at 0 */
while (true) {
    while (compare_and_swap(&bolt, 0, 1) == 1) /* busy wait */ ;
    /* critical section */
    bolt = 0;
    /* remainder */
}
```

- **Advantages:** works for any number of processes on one or many processors; simple; supports many critical sections.
- **Disadvantages:** **busy waiting** wastes CPU time; **starvation** is possible; **deadlock** is possible (a low-priority process in the critical section is pre-empted by a high-priority one that spins forever).

Exam answer template: Mutual exclusion and critical section (7 marks)

- Define critical section (shared-resource code) and mutual exclusion (only one in the critical section).
- Show the entry / critical / exit / remainder structure and the echo() race condition as the reason.
- List 3 ways (hardware, OS/language, software) and 3-4 of the requirements.

5.4 Semaphores

[🔥 HIGH-YIELD: Frequently Asked] F22 Q3 (7 marks): "counting semaphore and binary semaphore"; basis for J23 Q4C, M24 Q3A, J25 Q4A

A **semaphore** is a special **integer variable** used for **signalling** between processes. A process can be forced to **stop at a specified place until it receives a signal**. Only **three operations** are allowed on it: **initialise** (to a non-negative number), **semWait** and **semSignal**.

Real-world analogy

A **parking lot with a counter at the gate** showing free spaces. A car arriving does **semWait**: if the count is above zero it decrements it and enters; if zero, it **waits in a queue**. A car leaving does **semSignal**: it increments the count and lets one waiting car in.

Code / pseudocode: semaphore primitives (counting semaphore, from your deck)

```
struct semaphore { int count; queueType queue; };

void semWait(semaphore s) {
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignal(semaphore s) {
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place P on the ready list */;
    }
}

// Both operations are ATOMIC (indivisible).
```

Reading the count

- **count > 0**: the number of processes that can still do semWait **without blocking**.
- **count = 0**: the next semWait will block.
- **count < 0**: the value's magnitude = **number of processes currently blocked** on the semaphore.

Binary vs counting semaphore (F22 Q3)

Point	Counting (general) semaphore	Binary semaphore
Values	Any integer (initialised to non-negative); can go negative in the textbook definition	Only 0 or 1
Operations	semWait / semSignal	semWaitB / semSignalB
Use	Control access to a pool of identical resources (n buffer slots, n printers); counting events	Mutual exclusion (a lock) and simple signalling
semWait behaviour	Decrement; block if value becomes negative	If value is 1 change to 0 and continue; if 0, block
semSignal behaviour	Increment; wake one blocked process if any	If a process is blocked, unblock it; otherwise set value to 1
Analogy	Parking lot with n spaces	A single key to one room

Mutual exclusion with a semaphore

Code / pseudocode: n processes protecting a critical section

```
const int n = /* number of processes */;
semaphore s = 1;           // binary use: 1 = free, 0 = taken

void P(int i) {
    while (true) {
        semWait(s);          // entry section
        /* critical section */;
        semSignal(s);        // exit section
        /* remainder */;
    }
}
```

 **Example: Trace with two processes A and B (s starts at 1)**

A: semWait -> s = 0, **A enters**. B: semWait -> s = -1, **B blocks**. A leaves: semSignal -> s = 0, **B is unblocked** and enters. B leaves: semSignal -> s = 1 (free again).

Strong semaphore: the blocked-process queue is **FIFO** (guarantees no starvation). **Weak semaphore:** the order of release is unspecified (starvation possible).

5.5 The Producer/Consumer Problem

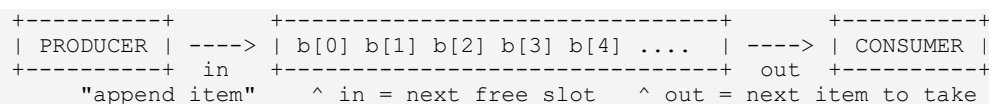
[💧 HIGH-YIELD: Frequently Asked] Asked in F22 Q3 (bounded buffer, counting semaphores), J23 Q4C (infinite buffer, binary semaphores), J25 Q4A (finite buffer, pseudo-code and discussion)

Problem statement: one or more **producers** generate data items and place them in a **buffer**. A **consumer** takes items out one at a time. **Only one process may access the buffer at any one time**. The consumer must not take from an **empty** buffer; with a bounded buffer, the producer must not add to a **full** buffer.

💡 Real-world analogy

A **sushi conveyor belt**. The chef (producer) places plates on the belt; the customer (consumer) takes plates off. The customer must wait when the belt is empty; the chef must wait when the belt is full. Two people must not grab the same spot at the same instant.

Figure: Producer / consumer with a buffer



Version 1: Infinite buffer, using BINARY semaphores (J23 Q4C)

Variables: **n** = number of items in the buffer. **s** = binary semaphore protecting the buffer (and **n**). **delay** = binary semaphore on which the consumer waits when the buffer is empty.

Code / pseudocode: infinite buffer with binary semaphores (correct solution)

```
int n; // items in buffer
binary_semaphore s = 1, delay = 0; // s = mutual exclusion, delay = 'buffer empty'

void producer() {
    while (true) {
        produce();
        semWaitB(s);
        append(); // put item in buffer
        n++;
        if (n == 1) semSignalB(delay); // buffer WAS empty: wake the consumer
        semSignalB(s);
    }
}

void consumer() {
    int m; // LOCAL copy of n
    semWaitB(delay); // wait for the first item
    while (true) {
        semWaitB(s);
        take(); // remove item from buffer
        n--;
        m = n; // copy n while still protected
        semSignalB(s);
        consume();
        if (m == 0) semWaitB(delay); // buffer now empty: sleep until producer signals
    }
}
```

⚠ Common mistake

Why the local variable `m`? If the consumer tested the shared `n` **after** `semSignalB(s)`, a producer might have added an item in between, and the consumer could wait on `delay` while the buffer is not empty (or consume a non-existent item). Copying `n` into `m` **inside the critical section** removes that race. (Your deck shows the faulty versions first and the fixes: a local variable, or a general semaphore.)

Version 2: Infinite buffer using a GENERAL (counting) semaphore

Simpler: let the semaphore `n` count the items directly.

Code / pseudocode: infinite buffer with counting semaphore `n`

```
semaphore n = 0, s = 1;           // n = number of items, s = mutual exclusion

void producer() {
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);             // one more item available
    }
}

void consumer() {
    while (true) {
        semWait(n);               // wait until at least one item exists
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
```

Version 3: BOUNDED (finite) buffer of size `N` with counting semaphores (F22 Q3, J25 Q4A)

Add a third semaphore `e` = number of **empty slots**, initialised to the buffer size.

Code / pseudocode: bounded buffer solution

```
const int sizeofbuffer = /* N */;
semaphore s = 1;           // mutual exclusion on the buffer
semaphore n = 0;           // number of FULL slots (items)
semaphore e = sizeofbuffer; // number of EMPTY slots

void producer() {
    while (true) {
        produce();
        semWait(e);         // wait for an empty slot (blocks if buffer full)
        semWait(s);         // enter critical section
        append();           // place item in buffer
        semSignal(s);       // leave critical section
        semSignal(n);       // one more full slot
    }
}

void consumer() {
    while (true) {
        semWait(n);         // wait for a full slot (blocks if buffer empty)
        semWait(s);
        take();             // remove item from buffer
        semSignal(s);
        semSignal(e);       // one more empty slot
        consume();
    }
}
```

🔧 Example: Trace with buffer size 2 (initially `n = 0`, `e = 2`, `s = 1`)

Producer makes item A: $e = 1$, append, $n = 1$. Producer makes item B: $e = 0$, append, $n = 2$.
 Producer makes item C: **semWait(e) blocks** (buffer full). Consumer: $n = 1$, take A, **semSignal(e)**
wakes the producer; $e = 1$... and so on.

The circular buffer (slides 322-325)

In practice the buffer is **circular**: the indexes wrap around using modulo, so space is reused. The **in** and **out** pointers chase each other round the ring.

Code / pseudocode: circular buffer of size N (busy-wait version from the deck)

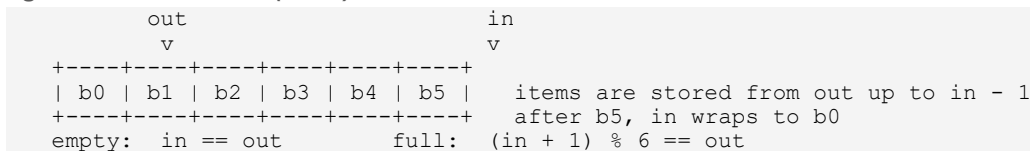
```

producer:
while (true) {
    /* produce item v */
    while ((in + 1) % n == out)
        /* do nothing: buffer full */;
    b[in] = v;
    in = (in + 1) % n;
}
// Full when (in+1)%n == out; empty when in == out. One slot is left unused.

consumer:
while (true) {
    while (in == out)
        /* do nothing: buffer empty */;
    w = b[out];
    out = (out + 1) % n;
    /* consume item w */
}

```

Figure: Circular buffer (N = 6)



What if the order of semaphore operations is wrong? (slide 321)

- **Swapping semSignal(s) and semSignal(n) in the producer:** harmless (it only makes the critical section slightly longer).
- **Swapping semWait(n) and semWait(s) in the consumer: fatal deadlock.** If the consumer enters its critical section holding s while the buffer is empty, no producer can ever get s to append, so the consumer waits forever on n .

Major disadvantage of semaphores: it is hard to produce correct designs because `semWait` and `semSignal` calls are **scattered** through programs, so their overall effect is hard to see.

Exam answer template: Producer/consumer using semaphores (6-8 marks)

- State the problem (producers, consumer, buffer; one access at a time; no take from empty; no add to full).
- Declare the semaphores and say what each does: s = mutual exclusion, n = items, e = empty slots.
- Write the producer and consumer pseudocode (use the bounded or infinite version the question names).
- Discuss: why e is needed only for bounded buffers; why order of waits matters (deadlock).

5.6 The Readers/Writers Problem

[★ ESSENTIAL FOR EXAM] M24 Q3A (8 marks): "Write the solution using semaphores where writers have priority"

A **shared data area** (a file or database) is accessed by many processes. Some only **read** (readers), others **write** (writers). Rules:

- **Any number of readers** may read at the same time.

- **Only one writer** at a time may write.
- If a **writer is writing**, **no reader** may read.

💡 Real-world analogy

A **library reference book**: any number of students can read it together, but when the librarian is correcting it (writing), nobody else may read or write until done.

Solution 1: readers have priority

Variables: **readcount** (readers currently reading), **x** (protects readcount), **wsem** (writer exclusion). The first reader locks writers out; the last reader lets them back in.

Code / pseudocode: readers have priority

```
int readcount = 0;
semaphore x = 1, wsem = 1;

void reader() {
    while (true) {
        semWait(x);
        readcount++;
        if (readcount == 1) semWait(wsem);    // first reader blocks writers
        semSignal(x);
        READUNIT();
        semWait(x);
        readcount--;
        if (readcount == 0) semSignal(wsem); // last reader lets writers in
        semSignal(x);
    }
}

void writer() {
    while (true) {
        semWait(wsem);
        WRITEUNIT();
        semSignal(wsem);
    }
}

// Problem: writers may STARVE if readers keep arriving.
```

Solution 2: writers have priority (the M24 question)

Once a writer wants to write, **no new reader may start reading**. Extra variables: **writcount**, semaphores **y** (protects writcount), **z** and **rsem** (hold new readers back while a writer is waiting).

Code / pseudocode: writers have priority

```
int readcount = 0, writcount = 0;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;

void reader() {
    while (true) {
        semWait(z);                // only one reader at a time queues on rsem
        semWait(rsem);             // blocked here if a writer is waiting/writing
        semWait(x);
        readcount++;
        if (readcount == 1) semWait(wsem);    // first reader blocks writers
        semSignal(x);
        semSignal(rsem);
        semSignal(z);
        READUNIT();
        semWait(x);
        readcount--;
        if (readcount == 0) semSignal(wsem);    // last reader releases writers
        semSignal(x);
    }
}

void writer() {
    while (true) {
        semWait(y);
```

```

        writecount++;
        if (writecount == 1) semWait(rsem);          // first writer blocks NEW readers
        semSignal(y);
        semWait(wsem);                               // exclusive access
        WRITEUNIT();
        semSignal(wsem);
        semWait(y);
        writecount--;
        if (writecount == 0) semSignal(rsem);        // last writer lets readers in
        semSignal(y);
    }
}

```

How writers' priority works, in plain words

- **rsem** is the "gate" for new readers. The **first waiting writer closes the gate**; the **last writer opens it**.
- Readers already inside finish (the last one signals **wsem**), then writers go one by one.
- **z** ensures only one reader waits at rsem, so a waiting writer can overtake the other readers.
- Trade-off: now **readers can starve** if writers keep arriving.

Exam answer template: Readers/writers (8 marks)

- State the three rules. Declare shared variables and semaphores and what each does.
- Write the reader and writer code of the version asked (writers' priority in M24; know readers' priority too).
- Close with the starvation trade-off of each version.

5.7 Monitors and Message Passing

[💡 **WILDCARD: Not asked yet, but in your deck**] Not asked in 5 papers; slides 326-338. A 1-page read is enough.

Monitors

A **monitor** is a programming-language construct: a software module with **local data**, **procedures and an initialisation sequence**. Key points: local data is accessible **only through the monitor's procedures**; a process enters by calling a procedure; **only one process can be inside the monitor at a time** (mutual exclusion is automatic); others wait.

For synchronisation, monitors use **condition variables** with two operations: **cwait(c)** suspends the caller on condition c (the monitor is released for others); **csignal(c)** resumes one process blocked on c (does nothing if none).

Real-world analogy

A monitor is a **bank teller counter with one window**: customers (processes) queue outside, only one is served at a time. If a customer needs a document not yet ready (condition), they step aside (cwait) and let the next customer in.

Advantage over semaphores: the compiler enforces mutual exclusion, so it is much **harder to get wrong**.

Message passing

Processes interact with two primitives: **send(destination, message)** and **receive(source, message)**. This provides both **communication** and **synchronisation**, and works on **distributed systems** as well as shared-memory machines.

Synchronisation style	Behaviour
Blocking send, blocking receive	Both wait until the message is delivered: a rendezvous .
Non-blocking send, blocking receive	Sender continues; receiver waits until a message arrives (most common).
Non-blocking send, non-blocking receive	Neither waits.
Direct addressing	send/receive name a specific process.
Indirect addressing	Messages go to a shared queue called a mailbox .

Code / pseudocode: mutual exclusion with message passing (blocking receive, mailbox 'box' starts with one message)

```
void P(int i) {
    message msg;
    while (true) {
        receive(box, msg);    // blocks until the 'token' is available
        /* critical section */
        send(box, msg);       // return the token
        /* remainder */
    }
}
```

Unit 5 quick revision

- Race condition (echo example) happens when shared data is accessed unsafely; fix = mutual exclusion.
- 3 degrees of awareness: unaware = competition; indirectly aware = cooperation by sharing; directly aware = cooperation by communication.
- Critical section = code touching shared resource. semWait decrements and blocks if negative; semSignal increments and wakes one.
- Binary semaphore (0/1, a lock) vs counting (resource pool).
- Bounded buffer: s (mutex), n (items), e (empty slots). Wrong order of waits = deadlock.
- Readers/writers: readcount + wsem; for writers' priority add writecount, y, z, rsem.

UNIT 6: Concurrency - Deadlock and Starvation (Chapter 6)

Deadlock is the **single most repeated topic** in your past papers: it appeared in **all 5 papers** and carries about **20% of in-scope marks**. It mixes theory, diagrams and two numerical styles (Banker's algorithm and resource allocation graphs). Master this unit first.

Unit map and priority

Definition, conditions (3 necessary + circular wait), deadlock vs livelock [🔥] - J22 Q4, F22 Q4, J23 Q4A, M24 Q4A.

Banker's algorithm / safe state numerical [🔥] - J23 Q5C, M24 Q3B (10 marks).

Resource allocation graph, consumable vs reusable, recovery [★] - J22 Q4, J25 Q3A (8 marks).

Prevention vs avoidance vs detection, prevention in detail [★] - F22 Q4.

Deadlock detection algorithm, dining philosophers [💡] - never asked, but on slides 384-397.

6.1 What Is Deadlock?

[🔥 **HIGH-YIELD: Frequently Asked**] J22 Q4, F22 Q4, J23 Q4A, M24 Q4A: "Define deadlock" is part of the question in 4 of 5 papers

Deadlock is the **permanent blocking of a set of processes** that either **compete for system resources** or **communicate with each other**. Each process in the set is waiting for an event that can only be caused by another process in the same set. **Deadlock is permanent** (nobody can ever proceed) and there is **no efficient general solution**.

Real-world analogy

Four cars arrive at a four-way crossing at the same moment. Each driver must wait for the car on their right to move, but that car is also waiting for the car on its right... Nobody can ever move. That is deadlock.

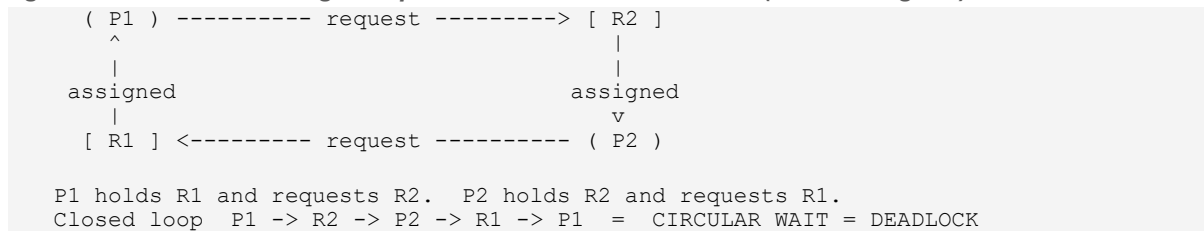
Example: Two processes, two resources (the classic)

Process P needs resources **A then B**; process Q needs **B then A**. Each needs both at the same time to finish.

Code / pseudocode: code that can deadlock

Process P	Process Q
-----	-----
Get A;	Get B;
Get B; <-- waits for Q	Get A; <-- waits for P
Release A; Release B;	Release B; Release A;
Bad interleaving:	P: Get A, Q: Get B, P: Get B (blocks), Q: Get A (blocks)
	P holds A, wants B. Q holds B, wants A. DEADLOCK.

Figure: Deadlock modelling: two processes and two resources (J22 Q4 diagram)



Your deck (slide 351) lists the **six possible execution orders** of this example. Only the orders in which **P gets A** and **Q gets B** before either gets its second resource deadlock:

#	Execution order and outcome
1	Q gets B, then A, then releases both. P later gets both. No deadlock.
2	Q gets B and A; P blocks on A; Q releases B and A; P resumes and gets both. No deadlock.
3	Q gets B, then P gets A . Q blocks on A, P blocks on B. Deadlock inevitable.
4	P gets A, then Q gets B . Q blocks on A, P blocks on B. Deadlock inevitable.
5	P gets A and B; Q blocks on B; P releases both; Q proceeds. No deadlock.
6	P gets A, B, releases both; Q gets both later. No deadlock.

Deadlock vs livelock vs starvation (M24 Q4A)

Term	Meaning and everyday picture
Deadlock	Processes are blocked forever , each waiting for a resource held by another. No state changes, no CPU is used. Picture: four cars frozen at a crossing.
Livelock	Processes are not blocked: they keep running and changing state in response to each other, but make no progress. They use CPU time. Picture: two people in a corridor who both step to the same side again and again, politely, forever.
Starvation	One process is perpetually denied a resource while other processes keep getting it. The system as a whole progresses. Picture: a customer never called at a counter because newcomers with higher priority keep arriving.

6.2 Conditions for Deadlock (Necessary and Sufficient)

[🔥 HIGH-YIELD: Frequently Asked] J22 Q4 (necessary and sufficient conditions), J23 Q4A ("three necessary conditions + the fourth essential condition"), M24 Q4A

Three **policy** conditions must be present for deadlock to be **possible**; a fourth, which is a **consequence** of the first three, makes it **actual**.

Condition	Meaning	Real-life picture
1. Mutual exclusion	Only one process may use a resource at a time.	A single-seat toilet / one printer.
2. Hold and wait	A process may hold allocated resources while waiting for others.	You keep your fork while waiting for the knife.
3. No pre-emption	A resource cannot be forcibly taken from the process holding it; only released voluntarily.	You cannot snatch the knife from someone's hand.

Condition	Meaning	Real-life picture
4. Circular wait	A closed chain of processes exists in which each holds a resource needed by the next, and the last waits for the first.	P1 waits for P2, P2 waits for P3, P3 waits for P1.

Necessary vs sufficient

- Conditions 1, 2, 3 are **necessary** for deadlock (without them, deadlock cannot happen) but **not sufficient**: they only make deadlock **possible**.
- All **four together** (1 + 2 + 3 + circular wait) are **necessary and sufficient** for deadlock to **exist** (for single-instance resources). That is why the 4th is called the fourth essential condition.
- Possible = conditions 1-3 present. Exists = conditions 1-4 present (slides 363-364).

Exam answer template: Deadlock definition and conditions (7-8 marks)

- Define deadlock (permanent blocking of processes competing for resources or communicating). Give the two-process/two-resource example with the diagram.
- List the 3 necessary conditions, then the 4th (circular wait), each with a one-line meaning and an example.
- State: 1-3 make it possible; all 4 make it actual. Add the deadlock vs livelock row if asked.

6.3 Resource Types: Reusable vs Consumable

[★ **ESSENTIAL FOR EXAM**] J25 Q3A (8 marks): "Differentiate consumable resources from reusable resources"

	Reusable resources	Consumable resources
Meaning	Used by one process at a time and not depleted by use. A process obtains it, uses it, then releases it for others.	Created (produced) and destroyed (consumed) . There is no limit on the number that may be produced.
Examples	CPU, I/O channels, main and secondary memory, devices, files, databases, semaphores.	Interrupts, signals, messages , information in I/O buffers.
How deadlock arises	Each process holds one resource and requests another .	A blocking receive where the awaited message is never sent.

Example: Reusable resource: memory (slide 355)

200 KB is available. P1: Request 80 KB ... later Request 60 KB. P2: Request 70 KB ... later Request 80 KB. If both processes reach their **second** request, P1 holds 80 and needs 60; P2 holds 70 and needs 80; only 50 KB is free. **Neither can proceed: deadlock.**

Example: Consumable resource: messages (slide 357)

P1: Receive(P2); ... Send(P2, M1); P2: Receive(P1); ... Send(P1, M2); If **receive is blocking**, both wait for a message the other will only send **after** receiving. Deadlock.

6.4 Resource Allocation Graph (RAG)

[★ ESSENTIAL FOR EXAM] J25 Q3A (8 marks): draw the RAG, say whether the system is deadlocked, describe recovery

A **resource allocation graph** is a **directed graph** that shows the state of processes and resources.

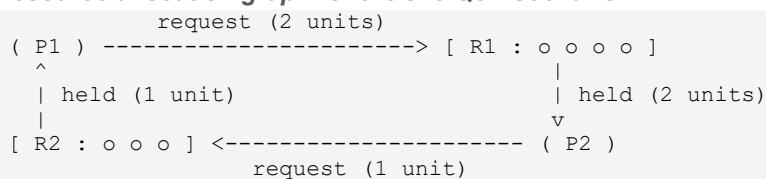
Symbol	Meaning
Circle (P)	A process .
Rectangle [R] with dots	A resource type ; each dot is one unit (instance) .
Arrow P to R (request edge)	Process requests a unit of R and is waiting.
Arrow R to P (assignment edge)	A unit of R is allocated to (held by) P.

Rules for reading a RAG: if the graph has **no cycle**, there is **no deadlock**. If it has a cycle and **each resource has a single unit**, there **is** deadlock. If resources have **multiple units**, a cycle is **necessary but not sufficient**: check whether any request can still be satisfied from the available units.

Example: The J25 Q3A question, solved step by step

System: processes **P1, P2**; resource **R1 has 4 units, R2 has 3 units**. P1 has requested 2 units of R1 and holds 1 unit of R2. P2 has requested 1 unit of R2 and holds 2 units of R1.

Figure: Resource allocation graph for the J25 Q3A scenario



Step	Calculation
Available R1	Total 4 - held by P2 (2) = 2 units free .
Available R2	Total 3 - held by P1 (1) = 2 units free .
Can P1's request be met?	P1 wants 2 units of R1; 2 are free, so yes . P1 gets them, runs to completion and releases its R1 and R2 units.
Can P2's request be met?	P2 wants 1 unit of R2; 2 are free, so yes .
Conclusion	There is a cycle in the picture (P1, R1, P2, R2), but because the resources have multiple units and both requests can be satisfied, the system is NOT in deadlock . (Safe sequence: P1 or P2 first, in either order.)

Common mistake

Do not say "cycle means deadlock" automatically. That is true only when every resource has **one unit**. With multiple units you must check **available units**. This is exactly the trap in J25 Q3A.

If it had been a deadlock, the question asks for the **recovery method**. See section 6.8 (abort all, roll back, abort one by one, pre-empt resources).

6.5 Deadlock Prevention

[★ **ESSENTIAL FOR EXAM**] F22 Q4 (7 marks): "Explain deadlock prevention in detail"

Prevention designs the system so that **at least one of the four conditions can never hold**. It is **conservative**: it undercommits resources to be safe.

Condition to break	How	Drawbacks
Mutual exclusion	Cannot be disallowed in general: some resources (printer, write access to a file) are inherently non-sharable .	Only possible for sharable resources (read-only files).
Hold and wait (indirect)	A process must request all its required resources at once and is blocked until all can be granted . It never holds one while waiting for another.	Inefficient (long waits); resources held idle for long; process may not know all needs in advance; starvation of processes needing popular resources.
No pre-emption (indirect)	If a process holding resources is denied a further request, it must release all it holds and request them again later. Or the OS pre-empts its resources and gives them to another process.	Works only for resources whose state is easily saved and restored (CPU, memory); not for printers; repeated rollbacks waste work.
Circular wait (direct)	Define a linear ordering of resource types . A process holding resource type R_i may later request only types that come after R_i in the order.	Slows processes and denies resource access unnecessarily; the order may not suit every program.

🔑 Example: Resource ordering removes the P and Q deadlock

Number the resources: **A = 1, B = 2**. Rule: always request in increasing number. Now **both P and Q must request A first, then B**. If P holds A, Q cannot get B first (it must get A first, which is held), so Q waits **without holding anything P needs**. The circular chain can never form.

Code / pseudocode: hold-and-wait prevention in pseudocode

```
// process must obtain EVERYTHING before starting its work
request_all(A, B, C);           // blocks until A, B and C are ALL available
work_with(A, B, C);
release_all(A, B, C);           // never holds one resource while waiting for another
```

6.6 Deadlock Avoidance and the Banker's Algorithm

[🔥 **HIGH-YIELD: Frequently Asked**] J23 Q5C (6 marks, describe Banker's algorithm with an example) and M24 Q3B (10 marks, need matrix + safe state + request check)

Avoidance allows the three conditions but **checks every resource request dynamically** and grants it only if the system **stays safe**. It needs to know in advance the **maximum** each process may claim.

Safe and unsafe states

- **Safe state**: there is **at least one sequence** of processes (a **safe sequence**) such that each can run to completion using the available resources plus those released by earlier processes.

- **Unsafe state:** no such sequence exists. **Unsafe does not mean deadlocked yet;** it means deadlock **may** occur. The system avoids entering unsafe states.

💡 Real-world analogy

A **bank with limited cash**. Each customer has a credit limit (maximum claim). The banker lends money only if, after the loan, he can still **pay every customer their full limit one at a time** in some order (customers repay when finished). If no such order exists, the loan is refused: the bank would be in an unsafe state.

Data structures (n processes, m resource types)

Name	Meaning
Resource vector (R)	Total units of each resource type in the system.
Available vector (V)	Units of each resource type currently not allocated .
Claim / Max matrix (C)	$C[i][j]$ = maximum units of resource j that process i may ever need.
Allocation matrix (A)	$A[i][j]$ = units of resource j currently held by process i.
Need matrix	Need = Max - Allocation (what each process may still request).

The safety algorithm (is this state safe?)

- 1. Set **Work = Available**. Mark every process as **not finished**.
- 2. Find a process i that is **not finished** and whose **Need[i] ≤ Work** (every component).
- 3. If found: **Work = Work + Allocation[i]** (it finishes and returns its resources); mark i finished; go to step 2.
- 4. If no such i exists: if **all processes are finished**, the state is **SAFE** (the order found is the safe sequence); otherwise **UNSAFE**.

The resource-request algorithm (can we grant this request?)

- 1. If **Request[i] > Need[i]**: error (process exceeded its maximum claim).
- 2. If **Request[i] > Available**: process must **wait** (resources not free).
- 3. Otherwise **pretend to allocate**: **Available -= Request**; **Allocation[i] += Request**; **Need[i] -= Request**.
- 4. Run the **safety algorithm** on the new state. If **safe**, **grant** the request. If **unsafe**, **undo** the pretend allocation and make the process wait.

Code / pseudocode: Banker's safety algorithm in pseudocode

```
bool is_safe(Available, Allocation, Need):
    Work = copy(Available)
    Finish[0..n-1] = false
    while true:
        find i with Finish[i] == false AND Need[i] ≤ Work
        if no such i: break
        Work = Work + Allocation[i]      // process i finishes, releases all
        Finish[i] = true
        append i to safe_sequence
    return (all Finish[i] are true)
```

Worked example: the May 2024 (M24 Q3B) question

System snapshot with four resource types A, B, C, D:

Process	Allocation (A B C D)	Max demand (A B C D)	Need = Max - Alloc	Notes
P0	3 0 1 1	4 1 1 1	1 1 0 0	4-3, 1-0, 1-1, 1-1
P1	0 1 0 0	0 2 1 2	0 1 1 2	0-0, 2-1, 1-0, 2-0
P2	1 1 1 0	4 2 1 0	3 1 0 0	4-1, 2-1, 1-1, 0-0
P3	1 1 0 1	1 1 1 1	0 0 1 0	1-1, 1-1, 1-0, 1-1
P4	0 0 0 0	2 1 1 0	2 1 1 0	2-0, 1-0, 1-0, 0-0

Available = (2, 0, 2, 0).

Question 1: What is the content of the Need matrix? See the **Need** column above (Need = Max - Allocation).

Question 2: Is the system in a safe state? Give a safe sequence. Apply the safety algorithm with Work = (2, 0, 2, 0):

Step	Process chosen	Check Need <= Work	New Work = Work + Allocation
1	P3	(0,0,1,0) <= (2,0,2,0) yes. (P0 needs B=1 > 0, P1 needs B=1 > 0, P2 needs A=3 > 2: not yet)	(2,0,2,0) + (1,1,0,1) = (3,1,2,1)
2	P0	(1,1,0,0) <= (3,1,2,1) yes	(3,1,2,1) + (3,0,1,1) = (6,1,3,2)
3	P1	(0,1,1,2) <= (6,1,3,2) yes	(6,1,3,2) + (0,1,0,0) = (6,2,3,2)
4	P2	(3,1,0,0) <= (6,2,3,2) yes	(6,2,3,2) + (1,1,1,0) = (7,3,4,2)
5	P4	(2,1,1,0) <= (7,3,4,2) yes	(7,3,4,2) + (0,0,0,0) = (7,3,4,2)

All five processes can finish, so the system **is in a SAFE state**. **Safe sequence: < P3, P0, P1, P2, P4 >** (other orders also exist, e.g. P3, P4, P0, P1, P2). Note the final Work (7, 3, 4, 2) equals the **total** resources.

Question 3: P1 requests (0, 0, 1, 0). Can it be granted immediately?

- **Check 1:** Request (0,0,1,0) <= Need[P1] = (0,1,1,2)? **Yes.**
- **Check 2:** Request (0,0,1,0) <= Available = (2,0,2,0)? **Yes.**
- **Pretend to allocate:** Available = (2,0,1,0). Allocation[P1] = (0,1,1,0). Need[P1] = (0,1,0,2).
- **Run the safety algorithm with Work = (2,0,1,0):** P3 (needs 0,0,1,0) fits, so Work = (3,1,1,1). P4 (2,1,1,0) fits, so Work = (3,1,1,1). P0 (1,1,0,0) fits, so Work = (6,1,2,2). P1 (0,1,0,2) fits, so Work = (6,2,3,2). P2 (3,1,0,0) fits, so Work = (7,3,4,2). **All finish.**

Conclusion: the new state is SAFE with sequence < P3, P4, P0, P1, P2 >, so the request CAN be granted immediately.

⚠ Common mistake

In the exam always show: (1) Need matrix, (2) the table of steps with Work after each step, (3) the safe sequence, and (4) for a request, **both checks and the pretend allocation** before the safety run. Marks are given for each step.

An unsafe example (to understand the idea)

🔑 Example: One resource type, 10 units

P1 holds 5 and may need 10 (needs 5 more). P2 holds 2, max 4 (needs 2 more). P3 holds 2, max 9 (needs 7 more). Available = $10 - 9 = 1$.

Safe? Work = 1. P1 needs $5 > 1$, P2 needs $2 > 1$, P3 needs $7 > 1$: nobody can finish. **Unsafe**. If instead 3 units were free (12 units in total), P2 could finish (Work = $3 + 2 = 5$), then P1 (Work = $5 + 5 = 10$), then P3: **safe**.

Two avoidance strategies and limitations

- **Process initiation denial:** do not start a process whose maximum demands could lead to deadlock.
- **Resource allocation denial** (Banker's algorithm): do not grant a request that could lead to an unsafe state.
- **Advantage:** less restrictive than prevention; no pre-emption needed.
- **Limitations:** the **maximum requirement of each process must be stated in advance**; processes must be **independent** (no synchronisation order constraints); a **fixed number of resources**; **no process may exit while holding resources**.

6.7 Deadlock Detection

[🔑 **WILDCARD: Not asked yet, but in your deck**] Not asked directly yet (recovery was, in J25 Q3A); slides 384-390

Detection does **not** restrict resource access. Resources are granted **whenever available**. The OS **periodically runs an algorithm** to detect a circular wait. It is **liberal**, in contrast to the conservative prevention strategies.

- **How often to check?** On every resource request (**early detection, simple algorithm** but **consumes a lot of CPU time**) or at intervals / when CPU utilisation drops.

The detection algorithm (marking algorithm, slide 387)

- **1. Mark** each process whose row in the Allocation matrix is all zeros (it cannot be in deadlock).
- **2.** Set a temporary vector **W = Available**.
- **3.** Find an **unmarked** process i whose **Request[i] ≤ W**. If none: **stop** - all unmarked processes are deadlocked.
- **4.** If found: **mark** it, **W = W + Allocation[i]**, and repeat step 3.

A deadlock exists **if and only if there are unmarked processes** at the end.

🔑 Example: Five resource types, four processes

Request: P1 (0 1 0 0 1), P2 (0 0 1 0 1), P3 (0 0 0 0 1), P4 (1 0 1 0 1). Allocation: P1 (1 0 1 1 0), P2 (1 1 0 0 0), P3 (0 0 0 1 0), P4 (0 0 0 0 0). **Available = (0 0 0 0 1)**.

Step 1: mark **P4** (allocation all zero). **Step 2:** $W = (0 0 0 0 1)$. **Step 3:** P3's request $(0 0 0 0 1) \leq W$, so **mark P3** and $W = (0 0 0 0 1) + (0 0 0 1 0) = (0 0 0 1 1)$. Now P1's request $(0 1 0 0 1)$ needs $B = 1 > 0$ and P2's request $(0 0 1 0 1)$ needs $C = 1 > 0$: **no more candidates**.

Result: **P1 and P2 are unmarked, so P1 and P2 are deadlocked.**

Recovery strategies once deadlock is detected (slides 389-390, J25 Q3A)

#	Strategy	Comment
1	Abort all deadlocked processes.	Most common in practice; simplest; but costs the most lost work.
2	Back up each deadlocked process to a previous checkpoint and restart all of them.	Needs rollback/restart mechanisms; the original deadlock may recur (non-determinism usually prevents it).
3	Successively abort deadlocked processes until deadlock no longer exists.	Re-run detection after each abort.
4	Successively pre-empt resources from deadlocked processes until deadlock no longer exists.	A pre-empted process must be rolled back to before it acquired that resource.

Selection criteria for which process to abort or pre-empt: **least processor time consumed so far; least output produced so far; most estimated time remaining; least total resources allocated so far; lowest priority.**

Prevention vs avoidance vs detection (F22 Q4)

	Prevention	Avoidance	Detection
Philosophy	Conservative; undercommits resources. Ensures one of the 4 conditions never holds.	Midway. Allows the 3 conditions but grants a request only if the state stays safe .	Very liberal. Grants requests whenever possible; checks periodically and recovers.
When it acts	At design time / request time (rules on how requests are made).	At every request at run time.	Periodically , after deadlock may have occurred.
Needs advance info	Sometimes (all resources at once).	Yes: maximum needs of each process.	No.
Pre-emption	Often needed.	Not needed.	Needed for recovery (losses).
Drawbacks	Inefficient; delays process start; blocks processes for long; may pre-empt unnecessarily.	Future needs must be known; processes may be blocked long; fixed resources.	Pre-emption/rollback losses ; detection overhead.
Example	Resource ordering; all-at-once requests.	Banker's algorithm.	Marking algorithm + abort/rollback.

💡 Real-world analogy

Prevention = building a bridge wide enough so two cars never meet. **Avoidance** = a traffic controller who lets a car on the single-lane bridge only if it can clear the bridge safely. **Detection** = letting cars on freely and sending a tow truck when a jam is found.

6.8 Dining Philosophers Problem

[💡 **WILDCARD: Not asked yet, but in your deck**] Never asked in 5 papers, but your deck gives it 5 slides + an exercise (slides 392-397)

Five philosophers sit at a round table. Between each pair of neighbours there is **one fork** (5 forks). A philosopher alternates between **thinking** and **eating**; to eat, they need **both** the left and the right fork. This models **processes competing for exclusive access to a limited set of resources**.

Figure: The dining philosophers

```

          [P0]
        f0 /   \ f4
        [P1]     [P4]
          \   /
          f1  f3 /
            [P2]--f2--[P3]
Fork f(i) lies between Pi and P(i+1).
Each Pi needs fork f(i) and fork f((i+1) mod 5) to eat.

```

Code / pseudocode: naive solution (can DEADLOCK)

```

semaphore fork[5] = {1,1,1,1,1};

void philosopher(int i) {
    while (true) {
        think();
        semWait(fork[i]);           // pick left fork
        semWait(fork[(i+1) % 5]);   // pick right fork
        eat();
        semSignal(fork[(i+1) % 5]);
        semSignal(fork[i]);
    }
}
// If all five pick up their left fork at the same time, each holds one fork
// and waits for the right one forever: hold-and-wait + circular wait = DEADLOCK.

```

Code / pseudocode: solution: allow at most four philosophers at the table (no deadlock, no starvation)

```

semaphore fork[5] = {1,1,1,1,1};
semaphore room = 4;           // at most 4 philosophers may try to eat

void philosopher(int i) {
    while (true) {
        think();
        semWait(room);         // enter the dining room
        semWait(fork[i]);
        semWait(fork[(i+1) % 5]);
        eat();
        semSignal(fork[(i+1) % 5]);
        semSignal(fork[i]);
        semSignal(room);       // leave the room
    }
}
// With only 4 at the table, at least one philosopher can always get both forks,
// so the circular wait can never close.

```

Other fixes: pick up both forks **atomically** (all-or-nothing, breaks hold-and-wait), or **number the forks** and always pick the lower-numbered first (resource ordering, breaks circular wait).

 Unit 6 quick revision

- Deadlock = permanent blocking of processes each waiting for a resource held by another in the set.
- Conditions: mutual exclusion, hold-and-wait, no pre-emption (necessary) + circular wait (with all three = sufficient).
- Livelock = running but no progress; starvation = one process perpetually denied while others proceed.
- Reusable (CPU, memory, files) vs consumable (messages, signals, interrupts). RAG: cycle + single units = deadlock; multiple units = check availability.
- Prevention (break a condition: all-at-once, pre-empt, ordering), avoidance (Banker's, safe state), detection (marking algorithm + recovery).

- Banker's: $\text{Need} = \text{Max} - \text{Alloc}$; safety loop ($\text{Need} \leq \text{Work}$, then $\text{Work} += \text{Alloc}$); for requests check $\leq \text{Need}$, $\leq \text{Available}$, pretend-allocate, re-run safety.
- Recovery: abort all; roll back to checkpoint; abort one by one; pre-empt resources.

FINAL REVISION SECTION: Master Cheat Sheets

Use this section the night before the exam. Every line points back to a full explanation in the units above.

A. The Diagrams You Must Be Able to Draw from Memory

[🔥 HIGH-YIELD: Frequently Asked] A labelled diagram is usually 2-3 of the 7 marks

#	Diagram	Used in question on	Section
1	Top-level computer components / registers	Computer system elements	1.1
2	Interrupt cycle + multiple-interrupt timeline	Interrupts	1.4
3	Cache read-operation flowchart	Cache memory	1.6
4	DMA block diagram + 3 configurations	I/O and DMA	1.7
5	SMP organisation	SMP / multiprocessor	1.8
6	Monolithic vs microkernel	Kernel	2.3
7	5-state and 7-state process models	Process states	3.2
8	Process image	Process image / PCB	3.3
9	Single- vs multi-threaded process; ULT vs KLT	Threads	4.1, 4.2
10	Resource allocation graph / deadlock model	Deadlock	6.1, 6.4

B. One-Page Formula and Procedure Sheet

Topic	Formula / procedure
Instruction format	Address bits = instruction bits - opcode bits. Memory = $2^{\text{address bits}}$. PC = address bits. IR = instruction bits.
Average memory access time	$T = H \times T_1 + (1 - H) \times (T_1 + T_2)$
Memory cost	Bits = bytes x 8. Cost = bits x cost-per-bit.
Semaphore	semWait: count--, block if count < 0. semSignal: count++, wake one if count <= 0.
Bounded buffer	s = 1 (mutex), n = 0 (items), e = N (empty slots). Producer: wait(e), wait(s), append, signal(s), signal(n). Consumer: wait(n), wait(s), take, signal(s), signal(e).
Banker's need	Need = Max - Allocation
Safety test	Work = Available. Repeat: find unfinished i with Need[i] <= Work; Work += Alloc[i]. Safe if all finish.
Request test	Request <= Need? Request <= Available? Pretend-allocate, then run the safety test.
Detection	Mark rows with zero allocation. W = Available. Find unmarked i with Request[i] <= W; mark it; W += Alloc[i]. Unmarked at end = deadlocked.

C. Quick Comparison Tables (one line each)

Pair	First	Second
Program vs process	Passive file on disk	Program in execution, with state and resources
Thread vs process	Dispatch unit; shares process resources	Resource-ownership unit; own address space
ULT vs KLT	App-managed; fast switch; one block stops all; no multi-CPU	Kernel-managed; per-thread blocking; multi-CPU; mode-switch cost
Multithreading vs multiprocessing	Many threads in one process (software)	Many processors (hardware)
Multiprocessor vs multicore	Several chips share memory/I/O	Several cores on one chip
Monolithic vs microkernel	All services in kernel (Linux, UNIX)	Minimal kernel + user servers (QNX, Mach, Minix)
Binary vs counting semaphore	0 or 1: lock	Any count: resource pool
Reusable vs consumable resource	Used and released (CPU, memory)	Produced and consumed (messages, signals)
Deadlock vs livelock	Blocked forever, no activity	Running but no progress
Prevention vs avoidance vs detection	Break a condition in advance	Grant only if safe / detect and recover
Programmed vs interrupt-driven vs DMA I/O	Poll the device	Interrupt per word / one interrupt per block

D. Practice Question Bank (with where to find the answer)

[🔥 HIGH-YIELD: Frequently Asked] Ordered by likelihood from the past-paper analysis

Likelihood	Question	Answer in
🔥 Very high	Define deadlock; state the necessary conditions; explain deadlock prevention.	6.1, 6.2, 6.5
🔥 Very high	Banker's algorithm: Need matrix, safe state and sequence, can a request be granted?	6.6
🔥 Very high	Write semaphore pseudocode for producer/consumer (bounded / infinite / circular).	5.5
🔥 Very high	Compare user-level and kernel-level threads with advantages and disadvantages.	4.2
🔥 High	What is a thread? Thread vs process; single vs multithreaded; multithreading vs multiprocessing.	4.1
🔥 High	Draw the 5-state / 7-state model; reasons for suspension.	3.2
🔥 High	Define OS; objectives; types; kernel; monolithic vs microkernel.	2.1 - 2.3
★ Medium-high	Multiprocessor vs multicore; SMP with diagram; advantages over uniprocessor.	1.8

Likelihood	Question	Answer in
★ Medium	Interrupts: why required, types, simple vs multiple.	1.4
★ Medium	Process image and PCB elements.	3.3
★ Medium	Readers/writers with semaphores (readers' vs writers' priority).	5.6
★ Medium	32-bit microprocessor numerical (memory, PC, IR, bus-width impact).	1.3
★ Medium	Process interaction: three degrees of awareness; mutual exclusion; critical section.	5.2, 5.3
💡 Wildcard	Dining philosophers; monitors; message passing; compare-and-swap; detection algorithm; memory-hierarchy numericals; cache; DMA.	1.5-1.7, 5.7, 6.7, 6.8

E. How to Write Each Type of Answer

Answer type	Strategy
6-8 mark theory	Define in 1-2 lines; give a diagram; a table or 4-6 bullet points; one example. About 1.5 to 2 pages.
Compare / differentiate	Draw a two-column table with at least 6 rows. End with a one-line conclusion.
Pseudocode	Declare variables and semaphores with initial values and a comment on each. Then the producer and the consumer. Add a two-line explanation.
Numerical (Banker's, access time, PC/IR)	Write the formula, then every step. Show intermediate vectors. State the final answer clearly (safe sequence, memory size, etc.).
Multi-part questions	Answer in labelled sub-parts that mirror the wording of the question, since each sub-part carries marks.

🔑 Final checklist before entering the exam hall

- Can I list the 4 deadlock conditions and break each one (prevention)?
- Can I do a Banker's problem end to end in under 10 minutes?
- Can I write the bounded-buffer producer/consumer and readers/writers code from memory?
- Can I draw the 7-state diagram, the SMP diagram and the process image?
- Can I give 6 differences between ULT and KLT?
- Do I have one definition ready for OS, kernel, process, thread, deadlock, semaphore, critical section, interrupt?